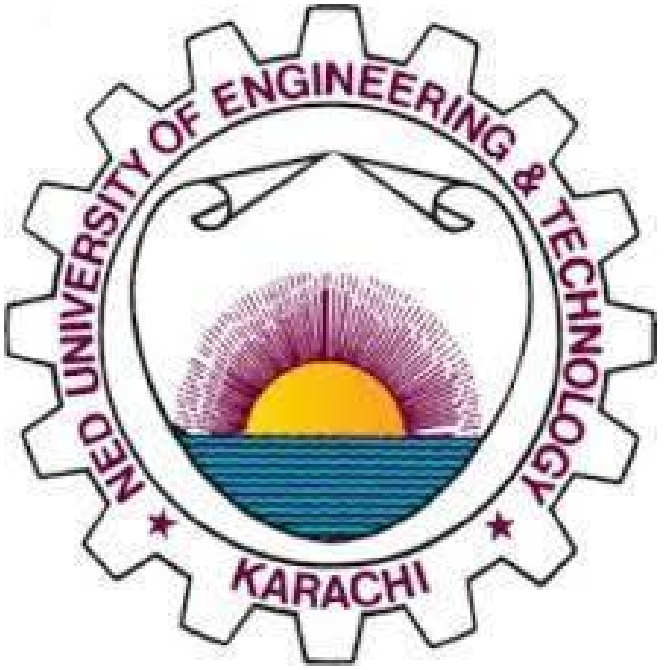
Practical Workbook

Software Construction & Development

(SE-312)



Name: _____

Seat #: _____

Batch: _____

Semester: _____

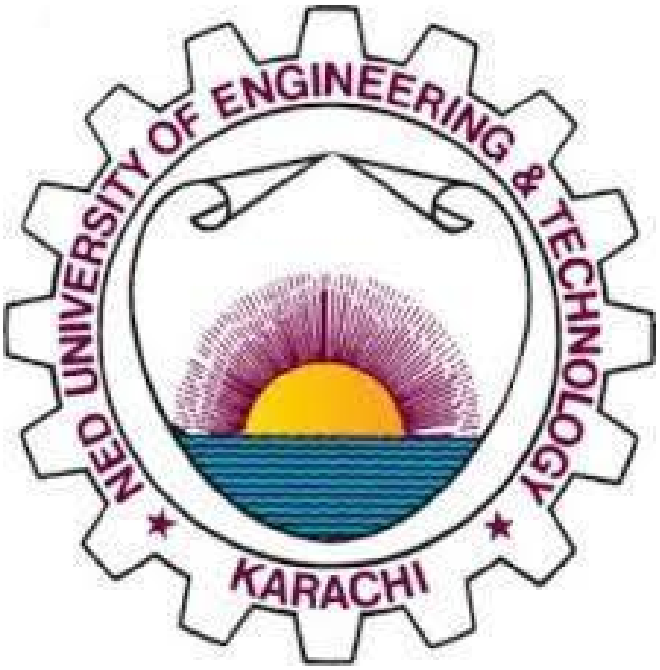
Year: _____

DEPARTMENT OF SOFTWARE ENGINEERING, NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Practical Workbook

Software Construction & Development

(SE-312)



Prepared By

Rafia Shakeel

Approved By

Prof. Dr. Shehnila Zardari
Chairperson

DEPARTMENT OF SOFTWARE ENGINEERING, NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Contents

Lab Session 01

Explore the usage of any documentation tool in Software Development Life Cycle (SDLC)

Software documentation

All large software development projects, irrespective of application, generate a large amount of associated documentation. A high proportion of software process costs is incurred in producing this documentation. Furthermore, documentation errors and omissions can lead to errors by endusers and consequent system failures with their associated costs and disruption. Therefore, managers and software engineers should pay as much attention to documentation and its associated costs as to the development of the software itself. The documents associated with a software project and the system being developed, have a number of associated requirements:

1. They should act as a communication medium between members of the development team.
2. They should be a system information repository to be used by maintenance engineers.
3. They should provide information for management to help them plan, budget and schedule the software development process.
4. Some of the documents should tell users how to use and administer the system.

Types of Software Documentation

Generally, the software project documentation produced falls into two classes:

1. ***Process documentation:*** These documents record the process of development and maintenance. *Plans, schedules, process quality documents and organizational and project standards are process documentation*. The major characteristic of process documentation is that most of it becomes outdated. Plans may be drawn up on a weekly, fortnightly or monthly basis. Progress will normally be reported weekly. Although of interest to software historians, much of this process information is of little real use after it has gone out of date and there is not normally a need to preserve it after the system has been delivered.

2. ***Product documentation:*** This documentation describes the product that is being developed. *System documentation describes the product from the point of view of the engineers developing and maintaining the system; user documentation provides a product description that is oriented towards system users.* Unlike most process documentation, it has a relatively long life. It must evolve in step with the product, which it describes. Product documentation includes user documentation, which tells users how to use the software product and system documentation, which is principally intended for maintenance engineers. See Figure 2.1 for different types of User Documents.

System documentation consists of the following:

- Software Requirement Specification (SRS)
- Design and Architecture
- Source Code Documents
- Testing Documents
- Formal Technical Reviews

User documentation includes the following:

- End-users Documentation
- System-admin Documentation

Document structure

The document structure is the way in which the material in the document is organized into chapters and, within these chapters, into sections and subsections. Document structure has a major impact on readability and usability. As with software systems, you should design document structures so that the different parts are as independent as possible. The IEEE standard for user documentation proposes that the structure of a document should include the components shown in Figure 2.2.

Component	**Description**
Identification data	Data such as a title and identifier that uniquely identifies the document.
Table of contents	Chapter/section names and page numbers.
List of illustrations	Figure numbers and titles
Introduction	Defines the purpose of the document and a brief summary of the contents
Information for use of the documentation	Suggestions for different readers on how to use the documentation effectively.
Concept of operations	An explanation of the conceptual background to the use of the software.

Procedures	Directions on how to use the software to complete the tasks that it is designed to support.
Information on software commands	A description of each of the commands supported by the software.
Error messages and problem resolution	A description of the errors that can be reported and how to recover from these errors.
Glossary	Definitions of specialized terms used.
Related information sources	References or links to other documents that provide additional information
Navigational features	Features that allow readers to find their current location and move around the document.
Index	A list of key terms and the pages where these terms are referenced.
Search capability	In electronic documentation, a way of finding specific terms in the document.

Fig 1.1: Suggested components in a software user document

Document Preparation

Document preparation is the process of creating a document and formatting it for publication. Figure 2.3 shows the document preparation process as being split into 3 stages namely document creation, polishing and production. The three phases of preparation and associated support facilities are explained in the figure below:

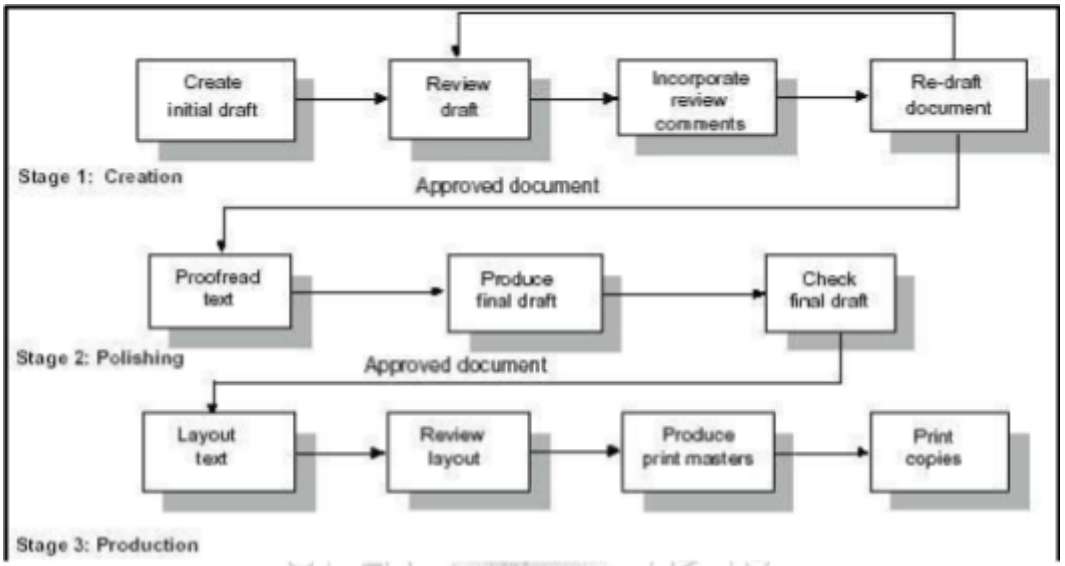


Fig 1.2: Stages of document preparation

LaTeX

LaTeX is a high -quality document preparation system; it includes features designed for the production of technical and scientific documentation. It allows users to very quickly tackle the more complicated parts of typesetting, such as inputting mathematics, creating tables of contents, referencing and creating bibliographies, and having a consistent layout across all sections. It is based on the WYSIWYM (what you see is what you mean) idea, meaning you only have focus on the contents of your document and the computer will take care of the formatting.

LaTeX is available as free software. To set up a basic TeX/LaTeX system, download and run the Basic MiKTeX Installer. TeXworks is used as editor for writing commands. We start off with a basic .tex file which contains LaTeX code. LaTeX uses control statements, which define how your content should be formatted. LaTeX compiler takes .tex file as input and convert it into a
.pdf file.

First Piece of LaTeX

```
\documentclass{article}

\begin{document}
First document. This is a simple example, with no extra
parameters or packages included.
\end{document}
```

The first line of code declares the type of document, known as the *class*. The class controls the overall appearance of the document. In this case, the class is article, the simplest and most common LATEX

class. Other types of documents you may be working on may require different classes such as **book** or **report**.
Then the contents of our document are written, enclosed inside the **\begin{document}** and **\end{document}** tags. This is known as the *body* of the document. You can start writing here and make changes to the text if you wish. To see the result of these changes in the PDF you have to compile the document.

The Preamble of a document

In the previous example the text was entered after the **\begin{document}** command. Everything in your **.tex** file *before* this point is called the **preamble**. In the preamble you define the type of document you are writing, the language you are writing in, the *packages* you would like to use (more on this later) and several other elements. For instance, a normal document preamble would look like this:

```
\documentclass[12pt, letterpaper]{article}
```

Below a detailed description of \documentclass:

\documentclass[12pt, letterpaper]{article}

As said before, this defines the type of document. Some additional parameters included in the square brackets brackets can be passed to the command. These parameters must be comma-separated. In the example, the extra parameters set the font size (**12pt**) and the paper size (**letterpaper**). Of course other font sizes (**9pt**, **11pt**, **12pt**) can be used, but if none is specified, the default size is **10pt**. As for the paper size other possible values are **a4paper** and **legalpaper.**

Adding a title, author and date

To add a title, author and date to our document, you must add three lines to the **preamble** (NOT the main body of the document). These lines are:

\title{First document}
\author{Hubert Farnsworth}
\thanks{funded by the Overleaf team}

This can be added after the name of the author, inside the braces of the **title** command. It will add a superscript and a footnote with the text inside the braces.

\date{September 2019}

Enter the date manually or use the command **\today** so the date will be updated automatically.

```
\documentclass[12pt, letterpaper, twoside] {article}

\title{First document}
\author{Hubert Farnsworth \thanks{funded by the Overleaf team}}
\date{September 2019}
```

Now a title, author and date are provided to the document, we can print this information on the document with the **\maketitle** command. This should be included in the **body** of the document at the place you want the title to be printed.

```
\begin{document}

\maketitle
We have now added a title, author and date to our first \LaTeX{}
document! \end{document}
```

Adding Comments

To make a comment in LATEX, simply write a **%** symbol at the beginning of the line as shown below:

```
\begin{document}

\maketitle
\pagenumbering {gobble}
We have now added a title, author and date to our first \LaTeX{}
document!
 % This line here is a comment. It will not be printed in the
document.
 \end{document}
```

Basic Formatting

- **Abstracts**

In scientific documents it's a common practice to include a brief overview of the main subject of the paper. In LATEX there's the **abstract** environment for this. The **abstract** environment will put the text in a special format at the top of your document.

```
\begin{document}
\newpage
\begin{abstract}
This is a simple paragraph at the beginning of the
document. A brief introduction about the main subject.
\end{abstract}
\end{document}
```

- **Paragraphs and Newlines**

When writing the contents of your document, if you need to start a new paragraph you must hit the "Enter" key twice (to insert a double blank line). Notice that LATEX automatically indents paragraphs. To start a new line without actually starting a new paragraph insert a break line point, this can be done by
\\ (a double backslash as in the example) or the \newline command.

```
\begin{document}
```

```
\begin{abstract}
This is a simple paragraph at the beginning of the document. A
brief introduction about the main subject.
\end{abstract}
Now that we have written our abstract, we can begin writing our
first paragraph.

This line will start a second Paragraph.
\end{document}
```

- **Chapters and Sections**

Commands to organize a document vary depending on the document type, the simplest form of organization is the sectioning, available in all formats.

```
\chapter{First Chapter}

\section{Introduction}

This is the first

section.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit.
Etiam lobortisfacilisis sem. Nullam nec mi et neque
pharetra sollicitudin. Praesent imperdietmi nec ante.
Donec ullamcorper, felis non sodales...

\section{Second Section}

Lorem ipsum dolor sit amet, consectetuer adipiscing elit.
Etiam lobortis facilisissem. Nullam nec mi et neque pharetra
sollicitudin. Praesent imperdiet mi necante...

\subsection{First Subsection}
Praesent imperdietmi nec ante. Donec ullamcorper, felis non
sodales...

\section*{Unnumbered Section}
Lorem ipsum dolor sit amet, consectetuer adipiscing elit.
Etiam lobortis facilisissem
```

The command **\section{}** marks the beginning of a new section, inside the braces is set the title. Section numbering is automatic and can be disabled by including a ***** in the section command as **\section*{}**. We can also have **\subsection{}**s, and indeed **\subsubsection{}**s. The basic levels of depth are listed below:

-1 **\part{part}**

0	**\chapter{chapter}**
1	**\section{section}**
2	**\subsection{subsection}**
3	**\subsubsection{subsubsection}**
4	**\paragraph{paragraph}**
5	**\subparagraph{subparagraph}**

Note that \part and \chapter are only available in report and book document classes.

· Bold, Italics and Underlining

Following are some simple text formatting commands.

- **Bold**: Bold text in LaTeX is written with the **\textbf{...}** command.
- *Italics*: Italicised text in LaTeX is written with the **\textit{...}** command.
- Underline: Underlined text in LaTeX is written with the **\underline{...}** command.

```
Some of the \textbf{greatest}
discoveries in \underline{science} were
made by \textbf{\textit{accident}}.
```

· Unordered lists

Unordered lists are produced by the **itemize** environment. Each entry must be preceded by the control sequence **\item** as shown below. By default the individual entries are indicated with a black dot, so-called bullet. The text in the entries may be of any length.

```
\begin{itemize}
  \item The individual entries are indicated with a black dot,
a so-called bullet.
\item The text in the entries may be of any length.
\end{itemize}
```

· Ordered lists

Ordered list have the same syntax inside a different environment. We make ordered lists using the enumerate environment:

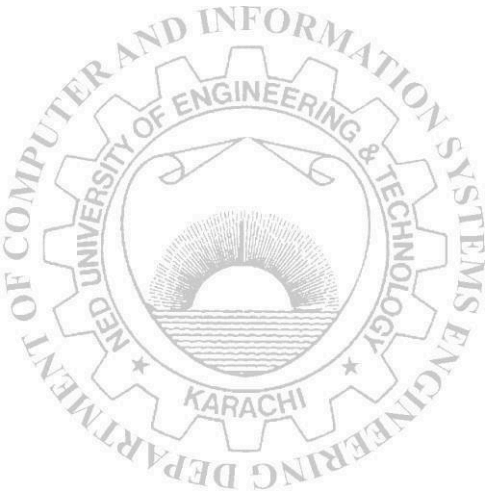
```
\begin{enumerate}
\item This is the first entry in our list
  \item The list numbers increase with each entry we add
\end{enumerate}
```

EXERCISES

1. Construct a Software Requirement Specifications (SRS) document for a Software System of your choice using LaTeX. Attach the printout of the .tex file and .pdf file.

2. Develop a brief document describing the basic functionality of an IoT based Health Monitoring System. Also include any block diagram/ image for explaining the overall functionality of the system. Use LaTeX. Attach the printout of the .tex and .pdf files. (Feel free to you use the other available formatting options in LaTeX).

3. Suppose that you are developing a calculator application and gathering requirements for it. Your calculator is able to solve some basic linear equations as well. Prepare an initial draft of requirement specification document, highlighting various examples of equations your system is able to solve. Use LaTeX. Attach the printout of the .tex and .pdf files.



Lab Session 02

Practice any project management tool to prepare a project plan

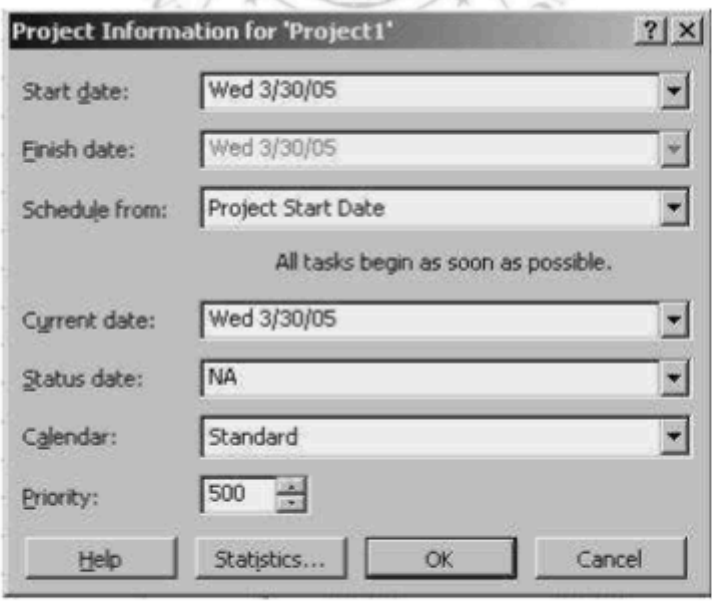
Project management

Project management is the process of planning, organizing, and managing tasks and resources to accomplish a defined objective, usually within constraints on time, resources, or cost. Sound planning is one key to project success, but sticking to the plan and keeping the project on track is no less critical. Microsoft Project features can help in monitoring progress and keep small slips from turning into landslides.

First let us understand some basic terms and definitions that will be used quite often:

Scope: of any project is a combination of all individual tasks and their goals.
Resources: can be people, equipment, materials or services that are needed to complete various tasks. The amount of resources affects the scope and time of any project.

You can create a Project from a Template File by choosing File > New fromthe menu. In the New File dialog box that opens, select the Project Templates tab and select the template that suits your project best and click OK. (You may choose a Blank Project Template and customize it)
Once a new Project page is opened, the Project Information dialog box opens.



***Tasks*:** They are a division of all the work that needs to be completed in order to accomplish the project goals.

STARTING A NEW PROJECT

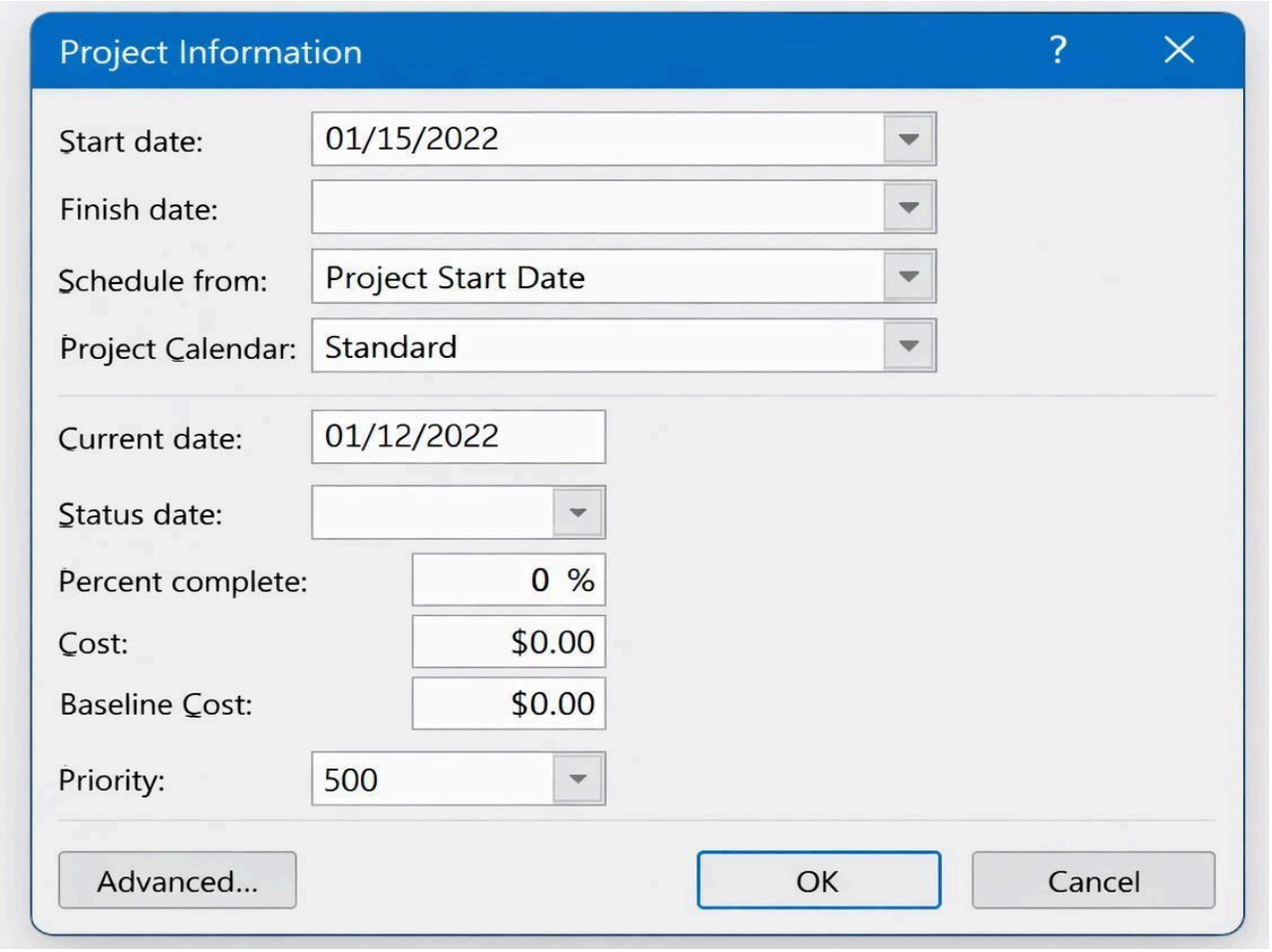


Fig 2.1: Project Information dialog box

Enter the start date or select an appropriate date by scrolling down the list. Click OK. Project automatically enters a default start time and stores it as part of the dates entered and the application window is displayed.

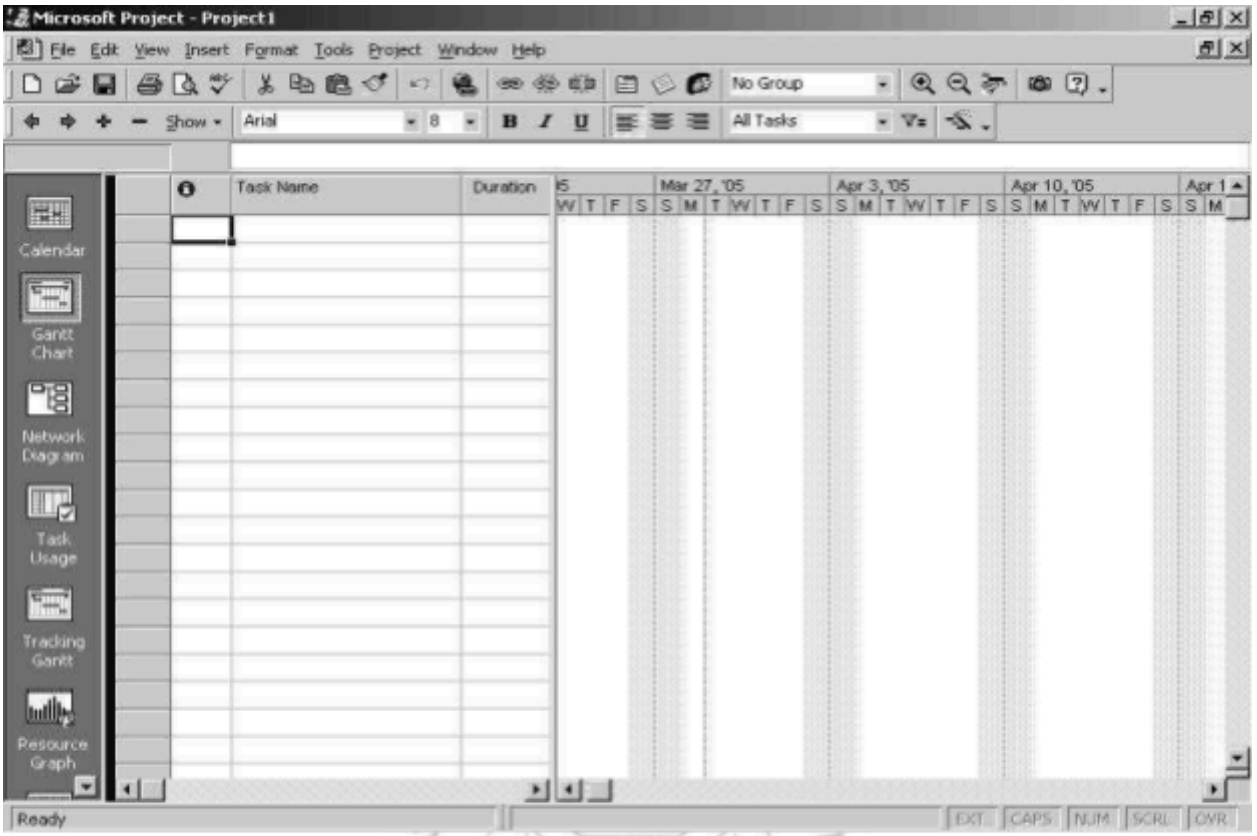


Fig 2.2: Main Window

Views

Views allow you to examine your project from different angles based on what information you want displayed at any given time. You can use a combination of views in the same window at the same time.

Project Views are categorized into two types:

- Task Views (5 types)
- Resource Views (3 types)

The Project worksheet is located in the main part of the application window and displays different information depending on the view you choose. The default view is the Gantt Chart View.

Tasks

Goals of any project need to be defined in terms of tasks. There are four major types of tasks:

1. *Summary tasks* - contain subtasks and their related properties
2. *Subtasks* - are smaller tasks that are a part of a summary task
3. *Recurring tasks* - are tasks that occur at regular intervals
4. *Milestones* - are tasks that are set to zero duration and are like interim goals in the project

- ***Entering Tasks and assigning task duration***

Click in the first cell and type the task name. Press enter to move to the next row. By default, estimated duration of each task is a day. But, there are very few tasks that can be completed in a day's time. You can enter the task duration as and when you decide upon a suitable estimate.

Double-clicking a task or clicking on the Task Information button on the standard toolbar opens the *Task Information box*. You can fill in more detailed descriptions of tasks and attach documents related to the task in the options available in this box.

To enter a *milestone*, enter the name and set its duration to zero. Project represents it as a diamond shape instead of a bar in the Gantt chart.

To copy tasks and their contents, click on the task ID number at the left of the task and copy and paste as usual.

You can enter *Recurring tasks* by clicking on Insert > Recurring task and filling in the duration and recurrence pattern for the task.

Any action you perform on a summary task - deleting it, moving or copying it apply to its subtasks too.

- ***Outlining tasks***

Once the summary tasks have been entered in a task table, you will need to insert subtasks in the blank rows and indent them under the summary task. This is accomplished with the help of the outlining tool.

Outlining is already active when you launch a project and its tools are found at the left end of the Formatting bar. To enter a subtask, enter the task in a blank cell in the Task Name column and click the Indent button on the Outlining tool bar. The Show feature in this toolbar is drop down tool that gives you an option of different Outline levels.

A summary task is outdented to the left cell border, is bold and has a Collapse (-) (Hide subtasks) button in front of it and its respective subtasks are indented with respect to it.

Advantages of Outlining:

- It creates multiple levels of subtasks that roll up into a summary task
- Collapse and expand summary tasks when necessary

- Apply a Work Breakdown structure
- Move, copy or delete entire groups of tasks

LINKS

Tasks are usually scheduled to start as soon as possible i.e. the first working day after the project start date.

Dependencies

Dependencies specify the manner in which two tasks are linked. Because Microsoft Project must maintain the manner in which tasks are linked, dependencies can affect the way a task is scheduled. In a scenario where there are two tasks, the following dependencies exist:

Finish to Start – Task 2 cannot start until task 1 finishes.
Start to Finish – Task 2 cannot finish until task 1 starts.
Start to Start – Task 2 cannot start until task 1 starts.
Finish to Finish – Task 2 cannot finish until task 1 finishes.

Tasks can be linked by following these steps:

1. Double-click a task and open the Task Information dialog box.
2. Click the predecessor tab.
3. Select the required predecessor from the drop down menu.
4. Select the type of dependency from drop down menu in the Type column.
5. Click OK.

The Split task button splits tasks that may be completed in parts at different times with breaks in their duration times.

CONSTRAINTS

Certain tasks need to be completed within a certain date. Intermediate deadlines may need to be specified. By assigning constraints to a task you can account for scheduling problems. There are about 8 types of constraints and they come under the flexible or inflexible category. They are:

- **As Late As Possible** – Sets the start date of your task as late in the Project as possible, without pushing out the Project finish date.
- **As Soon As Possible** – Sets the start date of your task as soon as possible without preceding the project start date.
- **Must Finish On** – Sets the finish date of your task to the specified date.
- **Must Start On** – Sets the start date of your task to the specified date.
- **Start No Earlier Than** – Sets the start date of your task to the specified date or later.
- **Start No Later Than** – Sets the start date of your task to the specified date or earlier.
- **Finish No Earlier Than** – Sets the finish date of your task to the specified date or later.
- **Finish No Later Than** – Sets the finish date of your task to the specified date or earlier.

Flexible constraints (demarcated by a red dot in Microsoft Project 2000) restrict scheduling to a great extent whereas flexible constraints (blue dot) allow Project to calculate the schedule and make appropriate adjustments based on the constraint applied.

Inflexible constraints can cause conflicts between successive and preceding tasks at times and you may need to remove such a constraint.

***To apply a constraint*:**

1. Open the Task Information dialog box.
2. Click the Advanced tab and open the Constraint type list by clicking on the drop-down arrow and select it.
3. Select a date for the Constraint and click OK.

UPDATE COMPLETED TASKS

If you have tasks in your project that have been completed as they were scheduled, you can quickly update them to 100% complete all at once, up to a date you specify.

1. On the View menu, click Gantt Chart.
2. On the Tools menu, point to Tracking, and then click Update Project
3. Click Update work as complete through and then type or select the date through which you want progress updated.
4. If you don't specify a date, Microsoft Project uses the current or status date.
5. Click Set 0% or 100% complete only.
6. Click Entire Project

RESOURCES

Once you determine that you need to include resources into your project you will need to answer the following questions:

- What kind of resources do you need?
- How many of each resource do you need?
- Where will you get these resources?
- How do you determine what your project is going to cost?

Resources are of two types - *work* resources and *material* resources.

Work resources complete tasks by expending time on them. They are usually people and equipment that have been assigned to work on the project.

Material resources are supplies and stocks that are needed to complete a project.
A new feature in Microsoft Project 2000 is that it allows you to track material resources and assign them to tasks.

Entering Resource Information in Project

The Assign Resources dialog box is used to create list of names in the resource pool.

To enter resource lists:

1. Click the Assign Resources button on the Standard Tool bar or Tools > Resources > Assign resources.
2. Select a cell in the name column and type a response name. Press Enter
3. Repeat step 2 until you enter all the resource names.
4. Click OK.

Resource names cannot contain slash (/), brackets [ ] and commas (,). Another way of defining your resource list is through the Resource Sheet View.

1. Display Resource Sheet View by choosing View > Resource Sheet or click the Resource Sheet icon on the View bar.
2. Enter your information. (To enter material resource use the Material Label field)

To assign a resource to a task:

1. Open the Gantt Chart View and the Assign Resources Dialog box.
2. In the Entry Table select the tasks for which you want to assign resources.
3. In the Assign Resources Dialog box, select the resource (resources) you want to assign.
4. Click the Assign button.

EXERCISE

1. Construct a project plan for a project who's SRS is documented in Lab session 1 (Exercise 1). Also attach its printout.

2. Construct a project plan for a project who's SRS is documented in Lab session 1 (Exercise 2). Also attach its printout.

NED University of Engineering & Technology
Department of Software Engineering
Course Code and Title

Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Understanding of SRS and Project Scope	Demonstrates complete understanding of SRS and project scope	Very good understanding with minor gaps	Adequate understanding	Limited understanding of requirements	Poor understanding	No understanding shown
Project Plan Structure	Project plan is fully structured with all components (tasks, timeline, resources, milestones)	Mostly complete structure	Basic structure present but missing some elements	Incomplete structure	Very limited structure	No project plan
Task Breakdown / Work Division	Clear and logical breakdown of project tasks	Mostly logical breakdown	Some tasks identified but incomplete	Tasks poorly identified	Minimal task breakdown	No task breakdown
Use of Planning Tool / Documentation	Planning tool used correctly with clear documentation	Minor errors in tool usage	Acceptable usage but incomplete	Limited use of planning tool	Poor documentation	No tool or documentation
Clarity and Presentation	Very clear, organized, and professional presentation	Mostly clear and organized	Acceptable clarity	Somewhat confusing	Poorly presented	No submission
Weighted CLO Score						
Remarks and Signature						

Lab Session 03

Carry out user view and structural view analysis for the suggested system: Use Case and Class Diagrams

Use case diagram

Use case diagrams are used to describe a set of actions (**use cases**) that some system or systems (**subject**) should or can perform in collaboration with one or more **external users** of the system (**actors**). Each use case should provide some observable and valuable result to the actors or other stakeholders of the system. (System) Use case diagrams are used to specify:

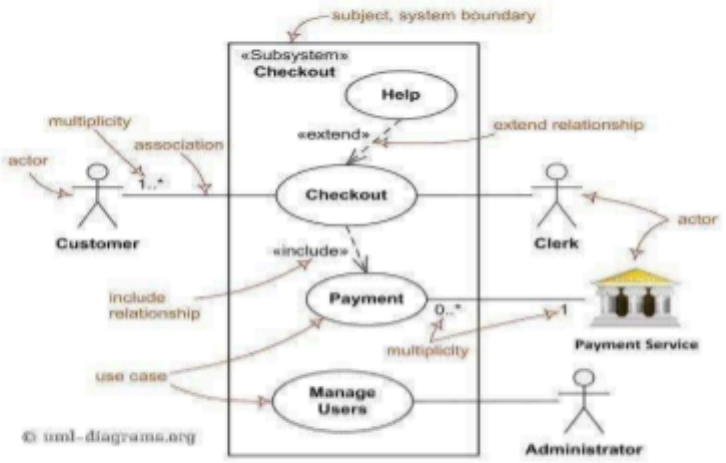
- (external) requirements, required usages of a system under design or analysis (subject) - to capture what the system is

Create Use Case Diagram

To create a Use Case Diagram in StarUML:

Select first an element where a new Use Case Diagram to be contained as a child.
Select **Model | Add Diagram | Use Case Diagram** in Menu Bar or select **Add Diagram | Use Case Diagram** in Context Menu.
To create use case subjects, actors, use cases, include and extendrelationship etc., select the desired option from toolbox and drag on the main diagram. Upon double clicking, all the respective options will be made available.

Major elements of the UML use case diagram are shown on the picture below.



*Major elements of UML use case diagram - **actor, use case, subject, include** and **extend** relationships.*

the functionality offered by a subject – what the system can do;
Requirements the specified subject poses on its environment - by defining how environment should interact with the subject so that it will be able to perform its services.

-
-

Class diagram

Class diagram is **UML structure diagram** which shows structure of the designed system at the level of **classes** and **interfaces**, shows their **features**, **constraints** and relationships **associations**, **generalizations**, **dependencies**, etc.

Create Class Diagram

To create a Class Diagram in StarUML:

To create classes, enumerations, association, composition, generalization and composition relationships, select the respective option from the toolbox.

First select an element where a new Class Diagram to be contained as a child.
Select **Model | Add Diagram | Class Diagram** in the Menu Bar or select **Add Diagram** Context Menu.

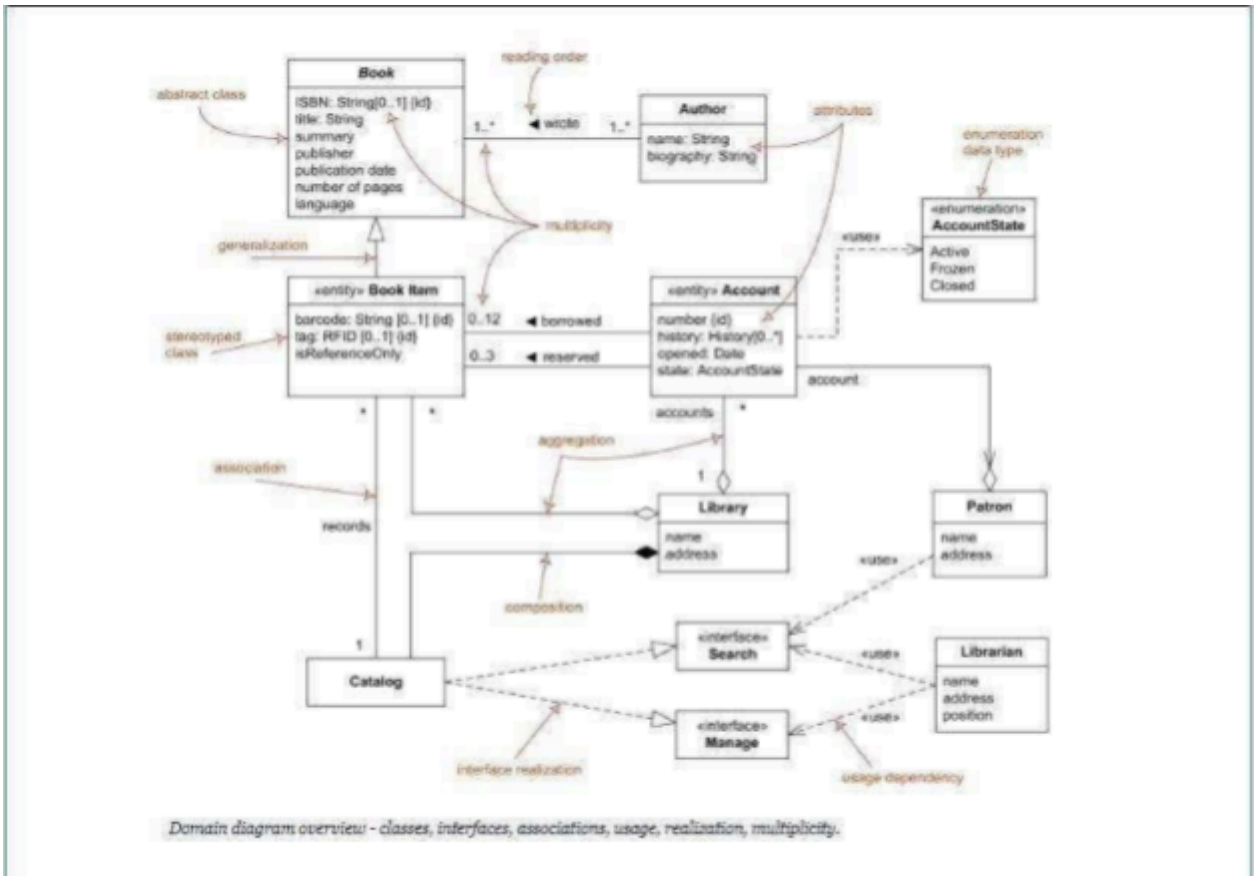


Fig 3.2 Major Elements of UML Class Diagram

Online Shopping System

Web Customer actor uses some web site to make purchases online. Top level **use cases** are **View Items**, **Make Purchase** and **Client Register**. View Items use case could be used by customer as top level use case if customer only wants to find and see some products. This use case could also be used as a part of Make Purchase use case. Client Register use case allows customer to register on the web site, for example to get some coupons or be invited to private sales. Note, that **Checkout** use case is **included use case** not available by itself - checkout is part of making purchase.

View Items use case is **extended** by several optional use cases - customer may search for items, browse catalog, view items recommended for him/her, add items to shopping cart or wish list. All these use cases are extending use cases because they provide some optional functions allowing customer to find item.

Customer Authentication use case is **included** in **View Recommended Items** and **Add to Wish List** because both require the customer to be authenticated. At the same time, item could be added to the shopping cart without user authentication.

UML Use Case Diagram Example

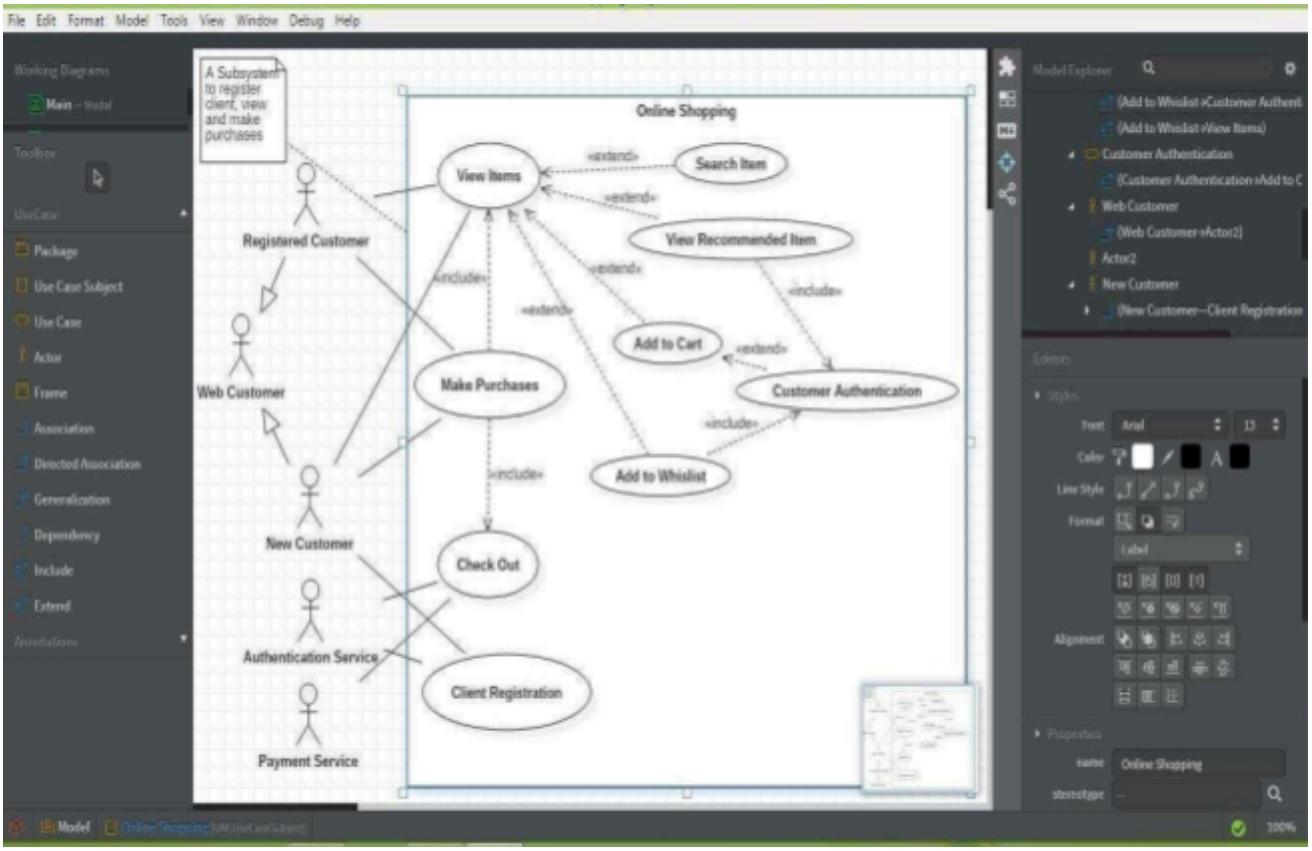
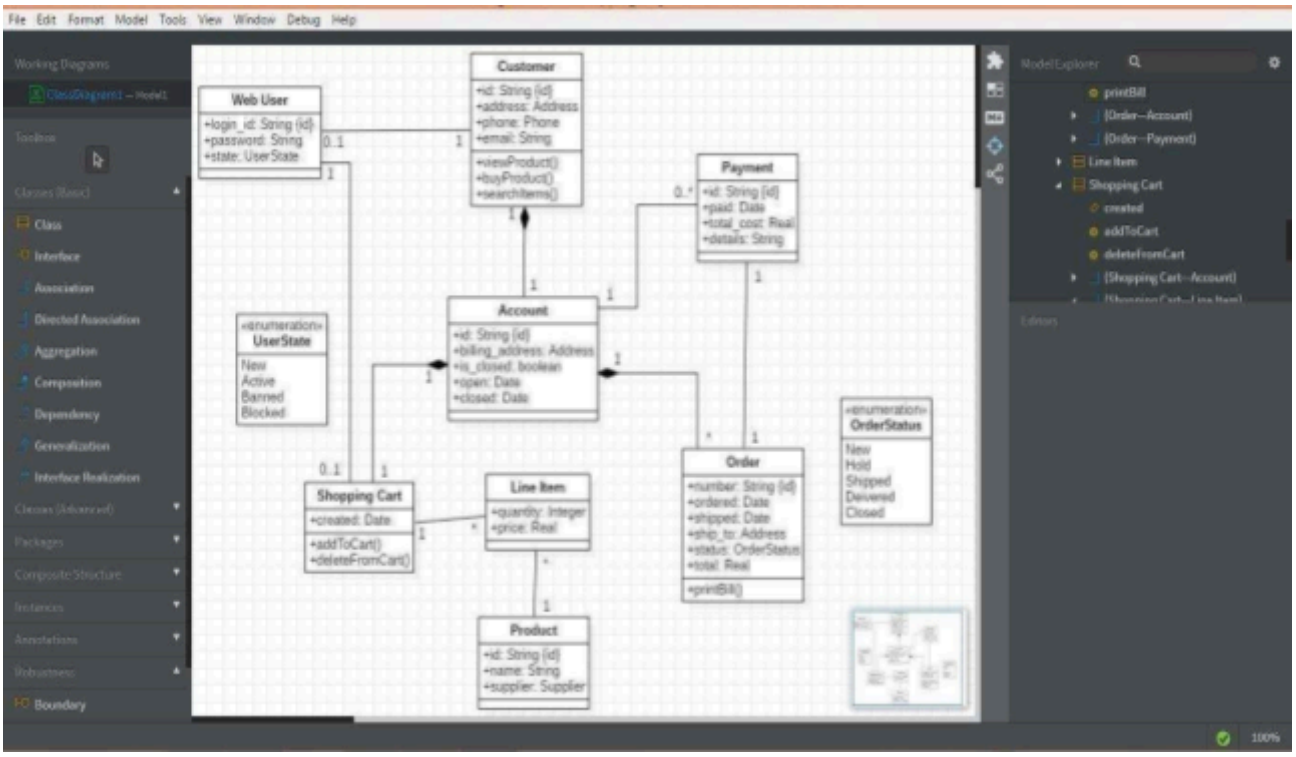


Fig 3.3 UML Use Case Diagram for Online Shopping System

UML Class Diagram Example



EXERCISES:

1. Construct use case and class diagram for an automated teller machine (ATM) or the automatic banking machine (ABM). Customer uses a bank ATM to check balances of his/her bank accounts, deposit funds, withdraw cash and/or transfer funds (use cases). ATM Technician provides maintenance and repairs to the ATM. Also attach printout.

2. For a hypothetical "Food Ordering App", identify possible classes, attributes and their operations. Also show the relationships among various classes using StarUML. Attach its printout.

3. In "Online Shopping System", Check Out use case includes several required uses cases. Web customer should be authenticated. It could be done through user login page, user authentication cookie ("Remember me"). Web site authentication service is used in all these use cases. Checkout use case also includes Payment use case which could be done by using external credit payment service. Extend the given use case diagram incorporating the given information. Also attach the

Lab Session 0 4

Practice behavioral view UML diagrams for the suggested system: State Chart, Sequence and Collaboration Diagrams

State chart diagram

State machine diagram is a **behavior diagram** which shows discrete behavior of a part of designed system through finite state transitions. State machine diagrams can also be used to express the usage protocol of part of a system.
The UML defines the Simple State, Composite State and Submachine state.

Simple State

A **simple state** is a state that does not have sub-states - it has no **regions** and it has no **submachine states**. Simple state is shown as a rectangle with rounded corners and the state name inside the rectangle. Simple state may have compartments. The compartments of the state are:

- name compartment
- internal activities compartment
- internal transitions compartment

Name compartment holds the (optional) name of the state, as a string. States without names are called **anonymous states** and are all considered distinct (different) states. Name compartments should not be used if a name tab is used and vice versa.

Internal activities compartment holds a list of internal actions or state (do) activities (behaviors) that are performed while the element is in the state. The activity label identifies the circumstances under which the behavior specified by the activity expression will be invoked. The behavior expression may use any attributes and association ends that are in the scope of the owning entity. For list items where the expression is empty, the slash separator is optional.
Several labels are reserved for special purposes and cannot be used as event names. The following are the reserved activity labels:

- entry (behavior performed upon entry to the state)
- do (ongoing behavior, performed as long as the element is in the state)
- exit (behavior performed upon exit from the state)

Internal transition compartment contains a list of internal transitions, where each item has the form as described for trigger. Each event name may appear more than once per state if the guard conditions are different. The event parameters and the guard conditions are optional. If the event has parameters, they can be used in the expression through the current event variable.

Fig 5.1 Simple state with name and internal activities compartments

Creating State Chart Diagram in StarUML

To create a State-chart Diagram:

1. Select first an element where a new Statechart Diagram to be contained as a child.
2. **Model | Add Diagram | Statechart Diagram** in Menu Bar or select **Add Diagram | Statechart Diagram** in Context Menu.
3. Toolbox contains all the required options namely simple states, internal activities (entry, do and exit), transitions, initial and final states etc. to create the required state chart diagram.

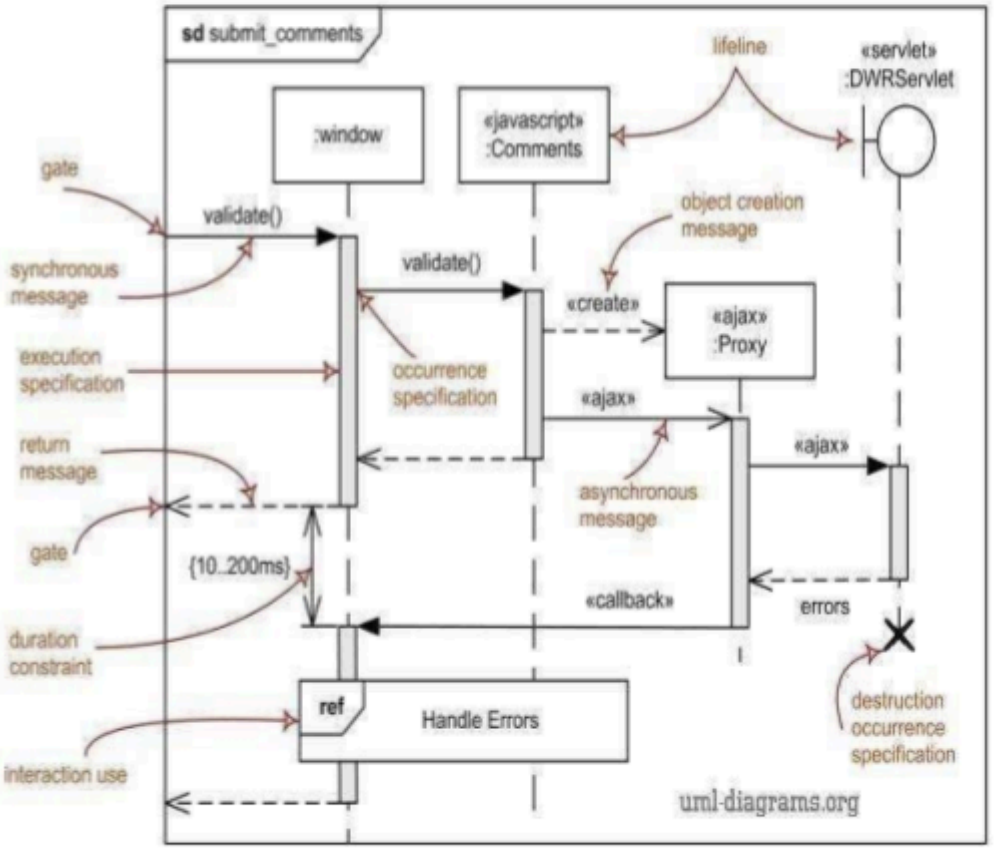
Sequence diagram

Sequence diagram is the most common kind of **interaction diagram**, which focuses on the **message** interchange between a numbers of **lifelines**.

Sequence diagram describes an interaction by focusing on the sequence of messages that are exchanged, along with their corresponding occurrence specifications on the lifelines.

, ,

Major elements of the sequence diagram are shown on the picture below.



Major elements of UML sequence diagram.

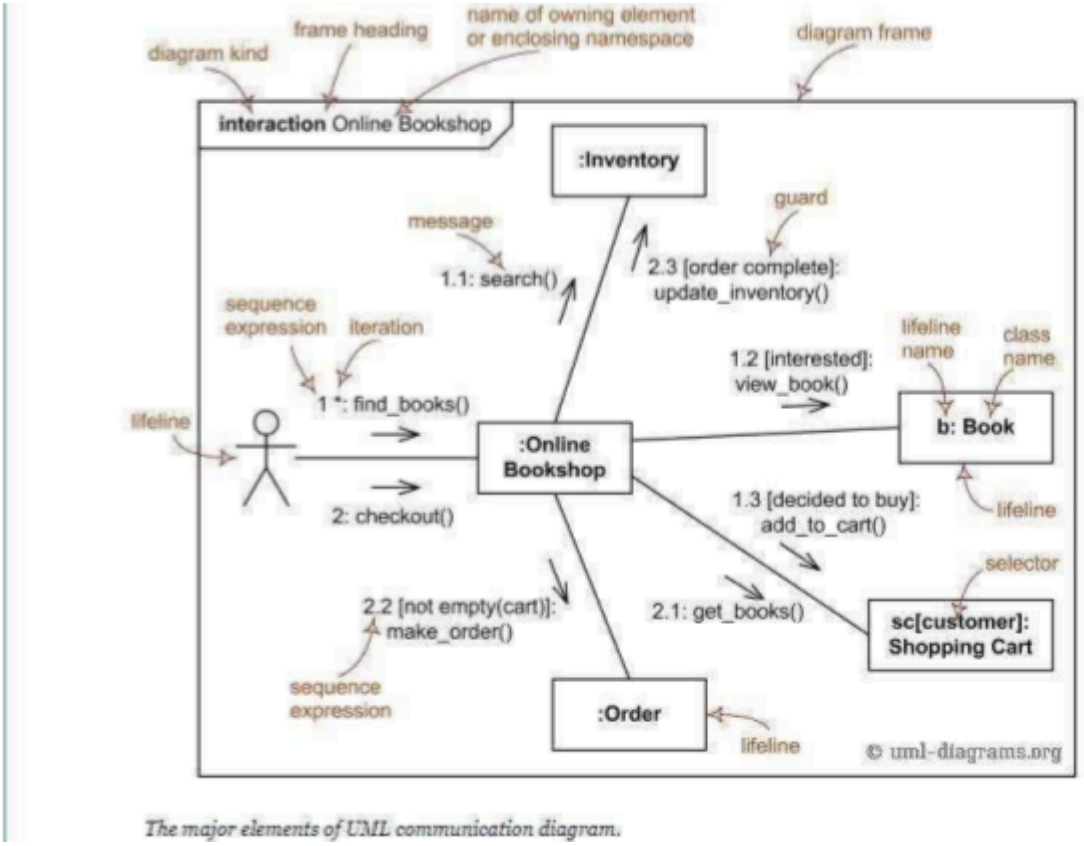
Create Sequence Diagram

To create a Sequence Diagram:

1. Select first an element where a new Sequence Diagram to be contained as a child.
2. Select **Model | Add Diagram | Sequence Diagram** in Menu Bar or select **Add Diagram | Sequence Diagram** in Context Menu.
3. All the major elements of UML sequence diagram (shown above) are available in the toolbox.

Collaboration or communication diagram

Communication diagram (called **collaboration diagram** in UML 1.x) is a kind of UML **interaction diagram** which shows interactions between objects and/or **parts** (represented as **lifelines**) using sequenced messages in a free-form arrangement.
Communication diagram corresponds (i.e. could be converted to/from or replaced by) to a simple **sequence diagram** without structuring mechanisms such as interaction uses and combined fragments. It is also assumed that **message overtaking** (i.e., the order of the receptions are different from the order of sending of a given set of messages) will not take place or is irrelevant.
The following nodes and edges are drawn in a UML communication diagrams: **frame**, **lifeline**, **and message**. These major elements of the communication diagram are shown on the picture below:



The major elements of UML communication diagram.

Create Communication/Collaboration Diagram

To create a Communication Diagram:

1. Select first an element where a new Communication Diagram to be contained as a child.
2. Select **Model | Add Diagram | Communication Diagram** in Menu Bar or select **Add Diagram | Communication Diagram** in Context Menu.
3. All the major elements of UML sequence diagram (shown above) are available in the toolbox.

Bank Automated Teller Machine (ATM) System

State Transition / State Chart Diagram

Purpose: An example of UML **behavioral state machine** diagram describing Bank Automated Teller Machine (ATM) top level state machine.
Summary: ATM is initially turned off. After the power is turned on, ATM performs startup action and enters **Self Test** state. If the test fails, ATM goes into **Out of Service** state, otherwise there is **triggerless transition** to the **Idle** state. In this state ATM waits for customer interaction.

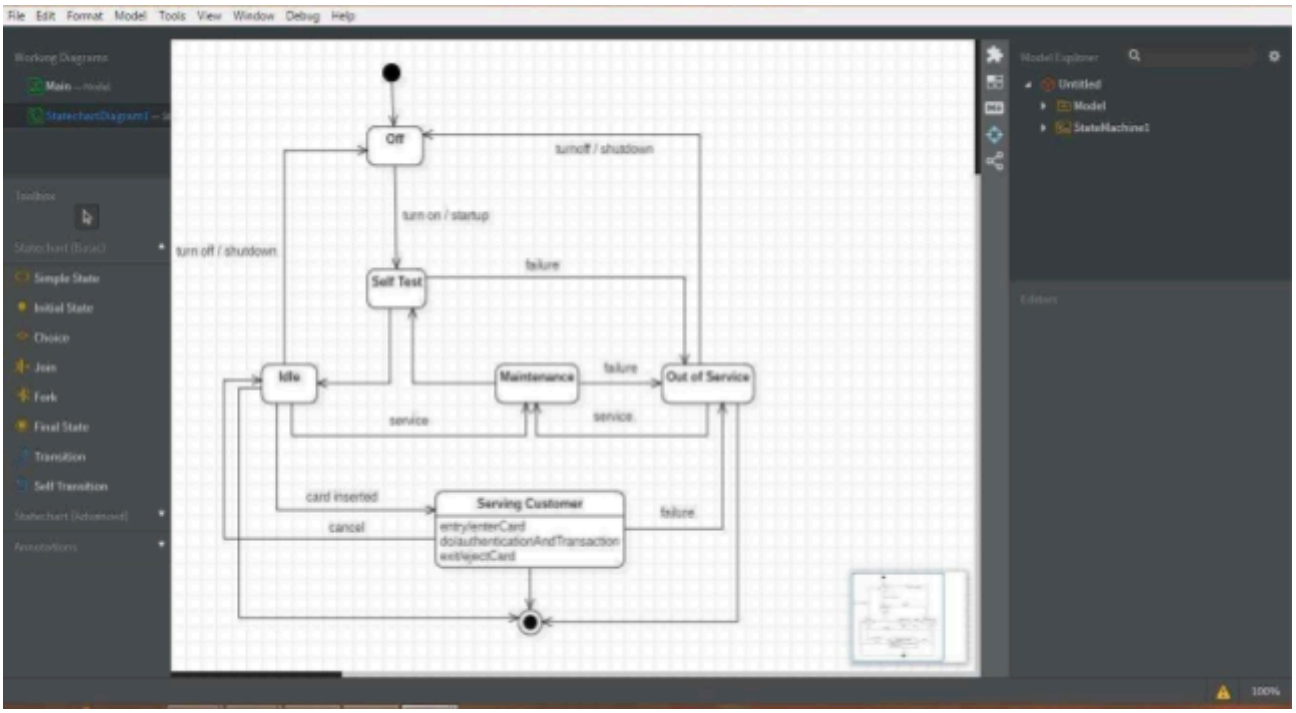


Fig 5.4 State Chart Diagram for ATM System

Sequence and Communication Diagram

Here, customer inserts his card in the ATM machine. The card is verified through bank and after authentication is done, customer is allowed to do transactions. He chooses to withdraw some cash. His request is processed and amount is debited from his account. The customer receives cash, card and receipt from the respective slots.

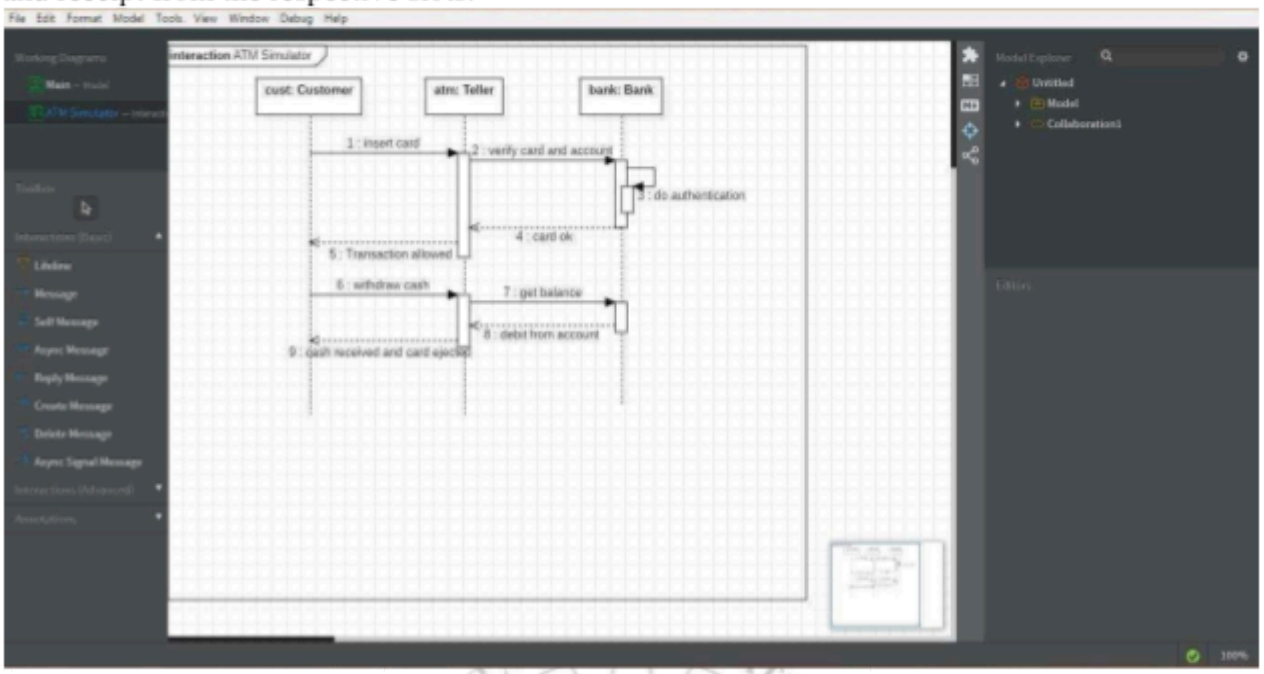


Fig 5.4 UML Sequence Diagram for ATM System

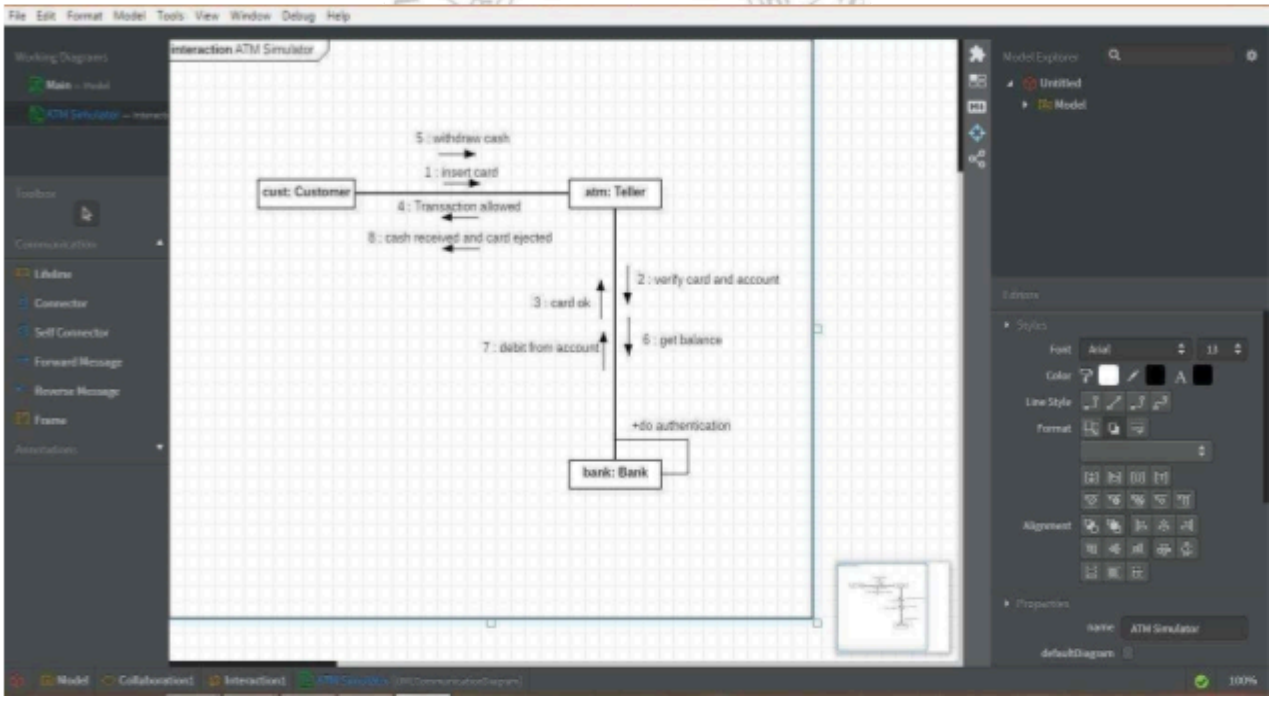


Fig 5.5 UML Collaboration Diagram for ATM System

EXERCISE

1. Create a State Chart Diagram for Microwave Oven Operation using StarUML. Also attach print out.

2. Create Sequence and Collaboration diagrams for Online Bookshop where the Web customer actor can search inventory, view and buy books online. Also attach the printout.

3. UML diagrams can also be created by means of variety of different online tools. Visual Paradigm is one of them. Create any three UML diagrams using this online tool for the ‘Online Bookshop System’. Attach print out

NED University of Engineering & Technology
Department of Software Engineering
Course Code and Title

Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Correctness of UML Diagrams	Diagrams are completely correct and follow UML standards	Minor notation mistakes	Mostly correct but some errors	Many UML errors	Very poor diagram correctness	No diagrams
State Chart Diagram (Microwave Oven)	Complete and accurate state transitions	Minor missing transitions	Basic states identified	Limited states shown	Incorrect state logic	Not attempted
Sequence & Collaboration Diagrams (Online Bookshop)	Clear interaction and message flow	Minor interaction issues	Basic interaction shown	Limited message flow	Incorrect diagram	Not attempted
Use of UML Tool (StarUML / Visual Paradigm)	Tool used effectively with proper formatting	Minor formatting issues	Acceptable usage	Limited tool usage	Poorly drawn diagrams	No tool used
Documentation and Explanation	Clear explanation of diagrams and system behavior	Mostly clear explanation	Some explanation provided	Minimal explanation	Poor explanation	No explanation
Weighted CLO Score						
Remarks and Signature						

Lab Session 05

User Interface Construction and Prototyping in Software Development

1. Objective

The objective of this lab is to introduce students to the concept of User Interface (UI) construction and early-stage software design. Before writing code, developers must first understand how users will interact with the software system. This is achieved by designing wireframes and simple prototypes that represent the structure and navigation of the system.

In modern software development practices, designing the interface before implementation is considered an essential activity. Wireframes help developers visualize the layout of a system, understand the placement of interface components, and ensure that the system provides a clear and user-friendly interaction flow.

Through this lab, students will learn how to design basic interface structures using prototyping tools. The lab also demonstrates how visual planning can reduce development errors, improve communication between development teams, and enhance the overall usability of a system.

This lab focuses on applying software construction principles to create structured and organized user interfaces for a simple software system.

2. Learning Outcomes

After completing this lab, students will be able to:

- Understand the importance of **interface design in software construction**
- Describe the concept of **wireframes and prototypes**
- Design a **basic user interface layout** for a simple software system
- Identify essential components of an interface such as headers, navigation menus, forms, and content areas
- Use a prototyping tool such as **Figma** to design interface structures
- Document interface design as part of the software development process

3. Introduction

Software development is a structured process that involves multiple stages such as **requirements analysis, design, construction, testing, and maintenance**. During the construction phase, developers transform system designs into working software through coding and integration.

However, before actual coding begins, developers must understand how users will interact with the system. This is where **User Interface (UI) design** becomes important.

A poorly designed interface can make a software system difficult to use, even if the underlying functionality works correctly. Therefore, software engineers often create **wireframes and prototypes** before implementation begins.

Wireframes provide a simplified visual representation of the system layout. They show how interface elements such as buttons, forms, images, and menus will be arranged on the screen.

By designing wireframes, developers can:

- Understand system navigation
- Identify required interface components
- Detect usability issues early
- Communicate design ideas with stakeholders

Wireframes are commonly used in the **design and construction stages** of the Software Development Life Cycle (SDLC). They serve as a blueprint for developers during implementation.

4. Background Theory

4.1 Software Construction

Software construction refers to the process of **building working software systems** by translating design specifications into executable code.

This phase typically includes:

- Coding
- Interface construction
- Integration of modules
- Debugging and testing

Although coding is the primary activity in software construction, developers must first design how the system will operate and how users will interact with it.

Therefore, **interface planning and design are essential activities before coding begins**.

4.2 User Interface (UI)

The **User Interface** is the part of the software system that allows users to interact with the application. It includes visual elements such as:

- Buttons
- Forms
- Text fields
- Navigation menus
- Icons

- Images

An effective user interface should be:

- Easy to understand
- Simple to navigate
- Visually organized
- Consistent across different pages

Good UI design improves **user satisfaction, productivity, and system efficiency**.

4.3 Wireframes

A wireframe is a simple visual layout of a user interface. It represents the structural design of a screen without focusing on detailed graphics or colors.

Wireframes usually consist of basic shapes such as:

- Rectangles representing images
- Lines representing text
- Boxes representing buttons
- Placeholder elements representing interface components

Wireframes help designers focus on layout and functionality rather than aesthetics.

4.4 Types of Wireframes

Wireframes can be categorized into three types:

1. Low-Fidelity Wireframes

Low-fidelity wireframes are simple sketches that show the general layout of the interface. They are quick to create and are often used during the early stages of design.

2. Mid-Fidelity Wireframes

Mid-fidelity wireframes provide more detail than low-fidelity wireframes. They include labels, interface components, and clearer structure.

3. High-Fidelity Wireframes

High-fidelity wireframes resemble the final design closely. They include detailed layouts and sometimes interactive elements.

In this lab, students will create **mid-fidelity wireframes**.

4.5 Prototyping

A **prototype** is a working model of a software system used to demonstrate how the system will function.

Prototypes may be:

- Static (non-interactive)
- Interactive (allowing navigation between screens)

Prototyping helps developers and stakeholders visualize the system before development begins.

5. Importance of Wireframing in Software Construction

Wireframing offers several advantages in software development:

1. Early Visualization

Wireframes allow developers to visualize the structure of the system before writing code.

2. Improved Communication

Design ideas can be easily communicated to team members and stakeholders.

3. Reduced Development Errors

By identifying design issues early, developers can avoid costly changes during implementation.

4. Better User Experience

Wireframes help designers focus on usability and navigation.

5. Faster Development Process

Clear design documentation allows developers to implement features more efficiently.

6. Tools for Wireframing and Prototyping

Several tools are available for designing wireframes and prototypes.

Common tools include:

- **Figma**
- **Balsamiq**
- **Adobe XD**
- **Canva**

For this lab, **Figma is recommended** because it is free, web-based, and easy to use.

7. Characteristics of a Good User Interface

A well-designed user interface plays an important role in the success of a software system. When users interact with software, they expect the interface to be clear, responsive, and easy to understand. Poor interface design can confuse users and reduce the overall effectiveness of the system.

The following characteristics are commonly found in good user interfaces.

7.1 Simplicity

A good interface should be simple and easy to understand. Users should be able to perform tasks without needing complex instructions. Unnecessary elements should be avoided because they can distract users and make the interface difficult to use.

7.2 Consistency

Consistency ensures that similar elements behave in the same way across different screens of the application. For example, navigation menus, buttons, and layout structures should appear in similar positions on different pages.

Consistency helps users learn the interface quickly and reduces confusion.

7.3 Visibility

Important actions and information should always be visible to users. Users should easily find buttons, menus, and navigation options. Hidden or unclear elements can make the system difficult to use.

7.4 Feedback

When users perform an action, the system should provide feedback. For example, when a user clicks a login button, the system should display a message indicating whether the login was successful or not.

Providing feedback improves user confidence and interaction.

EXERCISE

Question 1 – Interface Layout Design

Using Figma / Canva, create a **simple wireframe layout** for any software system of your choice (for example: Online Store, Student Portal, or Library System).

Your interface should include the following components:

- Header section
- Navigation menu
- Main content area
- Footer section

Use shapes and text to represent these components and attach a screenshot of your design in the lab report.

Question 2 – Login Page Interface Design

Using Figma, design a **Login Page interface** for the same system created in Question 1. Your design should include the following elements:

- Username input field
- Password input field
- Login button
- “Forgot Password” option
- “Sign Up” or “Create Account” option

Arrange these components properly to create a simple and clear login interface. Attach the screenshot of the design in the lab report.

Lab Session 06

Introduction and Working Of SOLID Design Principles in Software Construction

Objective

To understand and apply SOLID design principles using real-world software scenarios. Students will learn how proper design decisions improve maintainability, scalability, and flexibility in software systems.

Introduction

In modern software development, building a system that only works is not enough, it must also be **easy to maintain, extend, test, and scale**. Real-world software systems such as hospital systems, banking apps, e-commerce platforms, and university portals continuously evolve as user needs, business rules, and technologies change. New features are added, policies are updated, and user loads increase.

If a system is poorly designed, even small changes can introduce bugs, break existing functionality, or require large portions of the system to be rewritten. This leads to higher development costs, longer delivery times, and frustrated developers and users.

This is where **design principles** become important. Design principles provide proven guidelines for structuring software so that it remains flexible and robust over time. They help developers organize code in a way that reduces complexity, improves readability, and minimizes dependencies between components.

Using proper design principles helps to:

- Reduce tight coupling between modules
- Improve code reusability
- Make debugging and testing easier
- Support future enhancements without major rewrites
- Increase system stability and lifespan

One of the most widely accepted sets of design guidelines in object-oriented programming is the **SOLID principles**. These principles focus on writing clean, modular, and adaptable code. Instead of solving only today's problem, SOLID helps developers design systems that can handle future changes gracefully.

In this lab, we explore SOLID principles through real-world scenarios to understand not only *what* these principles are, but also *why* they are essential in professional software development.

1. Single Responsibility Principle (SRP)

Definition

A class should have only one responsibility and one reason to change.

Scenario: Hospital Management System

A hospital develops a system to manage patient records. Initially, the system is small. A developer creates a single class called PatientManager that:

- Stores patient details
- Calculates billing
- Sends appointment reminders via

email At first, everything works fine.

Problem Over Time

After a few months:

1. **Billing rules change** due to new insurance policies.
   → Billing logic must be updated.
2. Hospital switches from **email to SMS reminders**.
   → Notification logic must be changed.
3. Government regulations require **data format changes**.
   → Data storage logic must be modified.

All these changes require editing the same class.

This causes:

- High risk of bugs
- One fix breaking another feature
- Difficult testing
- Developer confusion

One class is doing too many jobs.

❌ Code (SRP Violation)

```
class PatientManager {
    void savePatient(){}
    void generateBill(){}
    void sendReminder(){}
}
```

✅ SOLUTION (Apply SRP)

Split Responsibilities

```java
class PatientRepository {
    void savePatient(){}
}

class BillingService {
    void generateBill(){}
}

class NotificationService {
    void sendReminder(){}
}
```

Explanation

Now:

- Billing change affects only BillingService
- Notification change affects only NotificationService
- Patient data remains safe

Teams can work independently.
Testing becomes easier.
System becomes modular.

2. Open/Closed Principle (OCP)

Definition

Classes should be open for extension but closed for modification.

Scenario: Online Shopping Platform

An e-commerce company runs seasonal campaigns. Initially, they offer:

- Student discounts
- Festival discounts

A developer writes a simple discount calculator.

Problem Over Time

The company grows. Marketing introduces:

- Black Friday discounts
- VIP customer discounts
- Loyalty rewards
- Flash sale discounts

Each time, developers modify the discount class.

Soon:

- Old discount logic breaks
- Bugs appear
- Testing takes longer
- Developers fear touching the

class The class becomes fragile.

❌ Code (OCP Violation)

```java
class Discount {
    double calculate(String type){
        if(type.equals("Student")) return 10;
        if(type.equals("Festival")) return 20;
        return 0;
    }
}
```

✅ SOLUTION (Apply OCP)

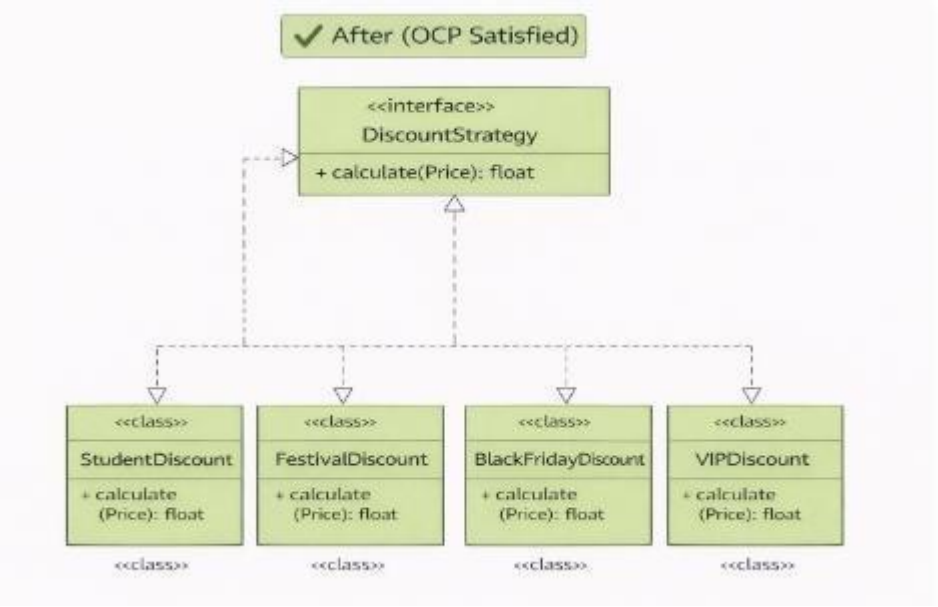
```java
interface Discount {
    double calculate();
}

class StudentDiscount implements Discount {
    public double calculate(){ return 10; }
}

class FestivalDiscount implements Discount {
    public double calculate(){ return 20; }
}
```

Explanation

Now when a new discount comes:
➡ Create a new class
➡ No modification to old code
➡ No risk to stability

System becomes future-proof.

UML Diagram



3. Liskov Substitution Principle (LSP)

Definition

Subclasses must work in place of their parent class without errors.

Scenario: Vehicle Management System

There is a company called **"GoFast"**. They provided all kinds of transportation services: taxis, buses, etc. The CEO hired a team of developers to create a **Vehicle Management System**.

Step 1: The Base Vehicle

The developers first created a base class called Vehicle. This class had a method drive() because every vehicle in the world can "drive" (or move).

```
class Vehicle {
    public void drive() {
        System.out.println("Vehicle is driving");
    }
}
```

At this point, everything seems fine. Taxi, Bus, and Car could inherit from this class:

```
java

class Car extends Vehicle { }

class Bus extends Vehicle { }
```

All cars and buses **work perfectly** with the Vehicle system. The company can call drive() on any Vehicle, and it behaves correctly.

Problem Over Time

Now, the company decides to deliver packages using **Delivery Drones**. But drones **don't drive**, they **fly**. The developer, trying to reuse the Vehicle class, writes:

```
java

class DeliveryDrone extends Vehicle {
    @Override
    public void drive() {
        throw new UnsupportedOperationException("Drone cannot drive!");
    }
}
```

At first, this seems okay because "we don't use drones like normal vehicles." But soon, a problem arises.

```
java

Vehicle v = new DeliveryDrone();
v.drive();   // Crash!
```

The system crashes because DeliveryDrone **violated the expectation set by** Vehicle. Any code expecting a Vehicle to "drive" should work with **all vehicles**, but now the drone breaks it.

This is exactly what **Liskov Substitution Principle** warns against..

✅ SOLUTION (Apply LSP)

The developers decides to redesign. They notice that **not all vehicles drive**. So they create **interfaces** or separate base classes:

```java
interface Vehicle {
}
```

(No methods — just a general type)

```
interface DriveableVehicle {
    void drive();
}

interface FlyableVehicle {
    void fly();
}
```

```java
class Car implements DriveableVehicle {
    public void drive() {
        System.out.println("Car driving");
    }
}

class Bus implements DriveableVehicle {
    public void drive() {
        System.out.println("Bus driving");
    }
}

class DeliveryDrone implements FlyableVehicle {
    public void fly() {
        System.out.println("Drone flying");
    }
}
```

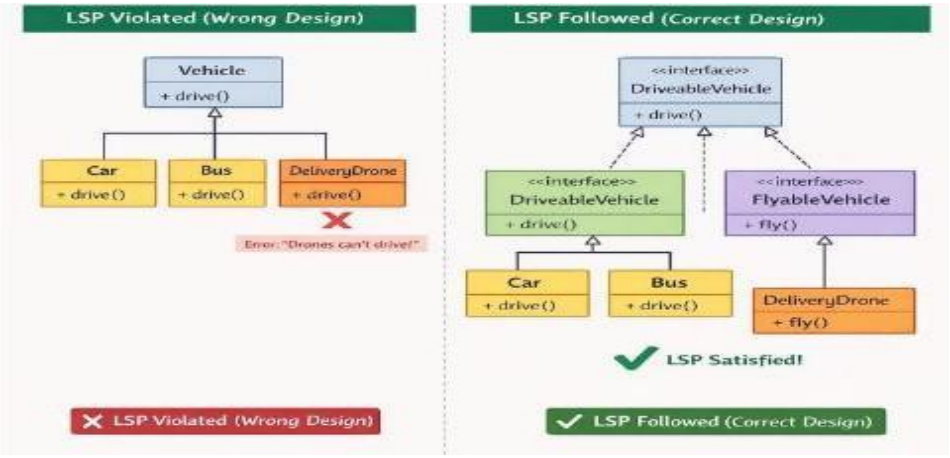
Explanation

Now, Car and Bus inherit from DriveableVehicle, and Drone inherits from FlyableVehicle:

Now, every subclass **fits the expectations of its parent**, and LSP is satisfied.

- DriveableVehicle can always be "driven."
- FlyableVehicle can always "fly."

UML Diagram



4. Interface Segregation Principle (ISP)

Definition

Clients shouldn't be forced to depend on methods they do not use.

If an interface is **too big (fat)**, classes are forced to implement methods that **don't make sense** for them.According to the interface segregation principle, you should break down "fat" interfaces into more granular and specific ones. Clients should implement only those methods that they really need.

Scenario: A Library for Cloud Services

Imagine you created a library called **CloudLink**, which makes it easy for apps to connect to cloud providers.

- **At first**, it only supported **Amazon Cloud**, so you included **all possible features**: compute, storage, database, AI, analytics, messaging, IoT etc.
- You created **one big interface** called CloudProvider that looked like this:

```java
interface CloudProvider {
    void compute();
    void storage();
    void database();
    void ai();
    void iot();
}
```

```java
class AmazonCloud implements CloudProvider {
    public void compute() { }
    public void storage() { }
    public void database() { }
    public void ai() { }
    public void iot() { }
}
```

Works perfectly for Amazon Cloud, because Amazon supports all these features.

Problem Over Time

Later, you want to support **TinyCloud**, a new cloud provider.

- Problem: TinyCloud **doesn't have AI, IoT, or messaging**.
- If your class tries to implement CloudProvider, you're **forced to add stubs** (empty methods) for features TinyCloud doesn't support:

❌ Code (ISP Violation)

```
class TinyCloud implements CloudProvider {

    public void compute() {
        System.out.println("Compute supported");
    }

    public void storage() {
        System.out.println("Storage supported");
    }

    public void database() { } // Empty
    public void ai() { }       // Empty
    public void iot() { }      // Empty
}
```

- Ugly, error-prone, and confusing for users.
- This is exactly the **fat interface problem** that the Interface Segregation Principle warns against.

✅ SOLUTION (Apply ISP)

Break the Interface into smaller pieces

Instead of one giant CloudProvider interface, we create **smaller, specific interfaces**:

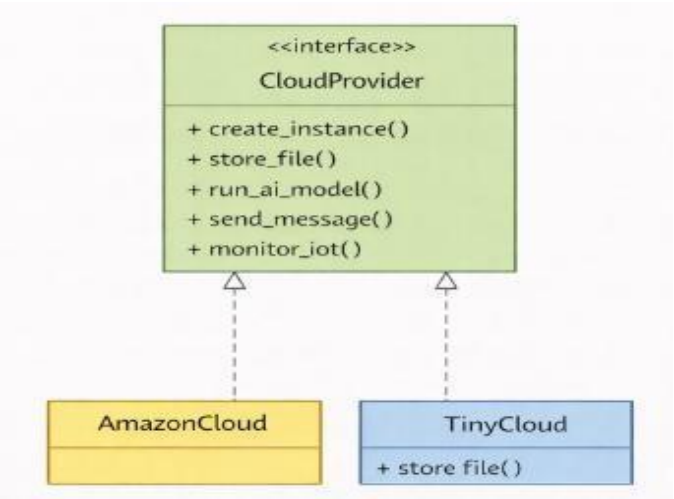
```
interface ComputeProvider {
    void compute();
}

interface StorageProvider {
    void storage();
}

interface DatabaseProvider {
    void database();
}

interface AIProvider {
    void ai();
}

interface IoTProvider {
    void iot();
}
```

```
class AmazonCloud implements ComputeProvider,
                             StorageProvider,
                             DatabaseProvider,
                             AIProvider,
                             IoTProvider {

    public void compute() { }
    public void storage() { }
    public void database() { }
    public void ai() { }
    public void iot() { }
}
```

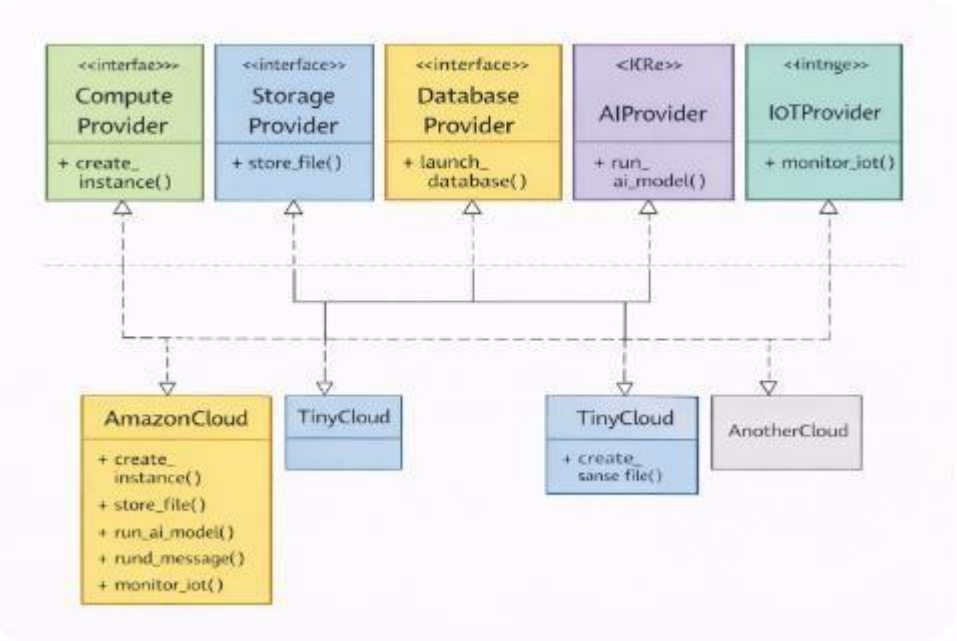
java

```
class TinyCloud implements ComputeProvider, StorageProvider {

    public void compute() {
        System.out.println("TinyCloud compute");
    }

    public void storage() {
        System.out.println("TinyCloud storage");
    }
}
```

Much cleaner! No empty methods, no confusion. Each cloud provider implements **only what makes sense**.



Before (ISP Violation):
One big CloudProvider interface forces all cloud services to implement methods they don't support, leading to empty or useless methods.



After (ISP Satisfied):
AmazonCloud

- Implements **all five interfaces** (compute, storage, database, AI, IoT)
- Because Amazon supports all features

TinyCloud

- Implements **only the interfaces it supports** (compute, storage)
- Ignores AI, IoT, database because TinyCloud doesn't support them

5. Dependency Inversion Principle (DIP)

Definition

High Level classes should not depend on low level classes. Both should depend on abstraction. Abstraction should not depend on details. Detail should depend on abstraction.

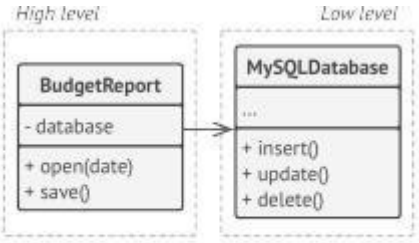
Scenario: The Budget Report and the Database

A company has a **BudgetReport** system that generates monthly financial reports.

They have a **high-level class**: BudgetReport → handles business logic.

They have a **low-level class**: MySQLDatabase → reads and writes data.

Initially, BudgetReport talks **directly to the database**:



BEFORE: a high-level class depends on a low-level class.

```
public class MySQLDatabase {

    public void insert(String data) {
        System.out.println("Insert into MySQL: " + data);
    }

    public void update(String data) {
        System.out.println("Update MySQL: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MySQL ID: " + id);
    }
}
```

```
public class BudgetReport {

    private MySQLDatabase database;

    public BudgetReport() {
        database = new MySQLDatabase(); // Direct dependency
    }

    public void saveReport(String report) {
        database.insert(report);
    }
}
```

Problem

Any change in Database (new database version, API changes) will **break BudgetReport**. High-level business logic **depends on low-level implementation details**.

This **violates DIP**.

✅ SOLUTION (Apply DIP)

Introduce Abstraction

Create an **interface** Database

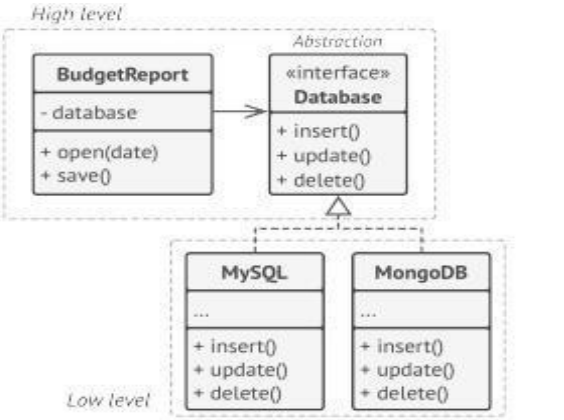
```
public interface Database {

    void insert(String data);
    void update(String data);
    void delete(int id);
}
```

```
public class MySQL implements Database {

    public void insert(String data) {
        System.out.println("Insert into MySQL: " + data);
    }

    public void update(String data) {
        System.out.println("Update MySQL: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MySQL ID: " + id);
    }
}
```

```
public class MongoDB implements Database {

    public void insert(String data) {
        System.out.println("Insert into MongoDB: " + data);
    }

    public void update(String data) {
        System.out.println("Update MongoDB: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MongoDB ID: " + id);
    }
}
```

```
public class BudgetReport {

    private Database database;

    public BudgetReport(Database database) {
        this.database = database;
    }

    public void saveReport(String report) {
        database.insert(report);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Database db = new MySQL(); // Can switch to MongoDB
        BudgetReport report = new BudgetReport(db);

        report.saveReport("January Budget");
    }
}
```



AFTER: low-level classes depend on a high-level abstraction.

Explanation

The **Database interface** contains common operations like:

- Read data
- Write data

These operations are basic functions that **any database can easily implement**, whether it is MySQL, MongoDB, or another database.

Because all databases support reading and writing data, they can implement this interface without difficulty.

The **BudgetReport class talks directly to the Database interface**, not to a specific database like MySQL or MongoDB.

This means:

- BudgetReport only knows about read and write operations
- It does not know which database is being used
- It does not depend on database details

EXERCISE

Task 1 — Applying SRP (Online Learning Platform Scenario)

An **online learning platform** manages:

- Student enrollment
- Course content delivery
- Progress tracking
- Email notifications to students

Currently, one class handles all these responsibilities.

Your Task

1. Explain why this violates the Single Responsibility Principle.
2. Identify at least three separate responsibilities.
3. Redesign the system by separating responsibilities into different classes.
4. Draw a UML class diagram of your improved design.
5. Provide sample code for your design.

Task 2 — Applying OCP (Movie Streaming Service Scenario)

A **movie streaming service** calculates subscription costs. Initially, it supports:

- Basic Plan
- Premium Plan

Later, the company introduces:

- Family Plan
- Student Plan
- Ad-supported Plan

The current pricing class must be edited every time a new plan is added.

Your Task

1. Explain why modifying the same class repeatedly is risky.
2. Redesign the system using OCP.
3. Draw a UML diagram showing how new plans can be added easily.
4. Write sample code supporting extensibility.

Task 3 — Applying LSP (Gaming System Scenario)

A **gaming system** has a base class GameCharacter with a method attack().

Some characters (like Warriors) can attack, but characters like **Healers** mainly restore health and do not attack.

Your Task

- Explain how forcing Healer to implement attack() violates LSP.
- Redesign the hierarchy properly.
- Draw UML diagrams before and after redesign.
- Provide corrected code.

Task 4 — Applying ISP (Smart Home System Scenario)

A **smart home system** controls:

- Smart Lights
- Smart Speakers
- Smart Door Locks

A large interface includes:

- playMusic()
- adjustBrightness()
- lockDoor()

Not all devices use all methods.

Your Task

1. Explain why this violates ISP.
2. Split the interface into smaller ones.
3. Draw UML diagrams.
4. Provide example implementation.

Task 5 — Applying DIP (E-Library System Scenario)

An **e-library system** stores books using a local database. Later, the library wants to support:

- Cloud storage
- External digital archives

The system directly depends on LocalDatabase class.

Your Task

1. Explain why direct dependency is problematic.
2. Redesign using DIP.
3. Draw UML showing abstraction usage.
4. Provide sample flexible code.

Task 6 — Mini Case Study (Integrated SOLID)

Consider a **Fitness Tracking App** that manages:

- Workout tracking
- Diet plans
- Notifications
- User progress reports

Your Task

1. Identify where at least three SOLID principles apply.
2. Suggest design improvements.
3. Draw UML diagram.
4. Provide sample code for one improvement.

NED University of Engineering & Technology
Department of Software Engineering
Course Code and Title

Criteria	(5)	(4)	(3)	(2)	(1)	(0)
Understanding of SOLID Principles	Excellent understanding of all SOLID principles (SRP, OCP, LSP, ISP, DIP)	Very good understanding with minor gaps	Adequate understanding of most principles	Limited understanding of principles	Poor understanding	No understanding
Application of Design Principles	Correct and well-structured design applying SOLID principles appropriately	Minor design issues but principles mostly applied correctly	Basic implementation of principles	Limited application of principles	Incorrect design or misuse of principles	Not implemented
UML Diagram Design	Accurate and well-structured UML diagrams clearly representing design	Minor UML issues but diagrams understandable	Acceptable diagrams with some missing details	Incomplete diagrams	Poor or confusing diagrams	No diagrams
Scenario Analysis	Correct identification of design problems and proper application of SOLID principles	Minor analysis gaps	Basic scenario analysis	Limited reasoning about design problems	Poor analysis	No analysis
Code / Implementation Quality	Code is clean, correct, modular, and follows SOLID principles	Minor coding issues	Acceptable implementation with basic structure	Limited functionality	Poor code quality	No code
Weighted CLO Score						
Remarks and Signature						

Lab Session 07

Introduction and Working of Design Pattern in Software Construction

Objective

To understand the concept, purpose, and working of design patterns and apply them to real software scenarios. Students will implement and analyze three commonly used design patterns to improve software structure, flexibility, and maintainability.

Introduction

In real-world software development, developers frequently encounter similar design problems. As software systems grow larger and more complex, managing code becomes difficult. Without proper design, systems become:

- Hard to modify
- Difficult to test
- Error-prone
- Rigid and tightly

coupled For example:

- A system may depend on too many classes
- One change may break many modules
- Code duplication increases
- Maintenance becomes expensive

To solve such recurring problems, software experts introduced **Design Patterns**.

What are Design Patterns?

A design pattern is a **general reusable solution to a common software design problem**.

It is not a finished code but a **template** that guides developers in structuring code.

Design patterns help in:

- Writing clean code
- Reducing coupling
- Increasing reusability
- Improving scalability
- Making communication easier among developers

Why Design Patterns are Important

Design patterns are important because they:

1) Promote Reusability

Instead of solving the same problem from scratch, we reuse proven solutions.

2) Improve Communication

If a developer says “We are using Singleton,” others immediately understand the design.

3) Reduce Development Risk

Patterns are tested and widely used.

4) Improve Maintainability

Code becomes easier to modify and extend.

5) Provide Best Practices

They represent industry-standard solutions.

Categories of Design Patterns

Design patterns are generally divided into:

1. Creational Patterns

Deal with object creation. ***Example***: Singleton

2. Structural Patterns

Deal with object relationships. ***Example***: Facade

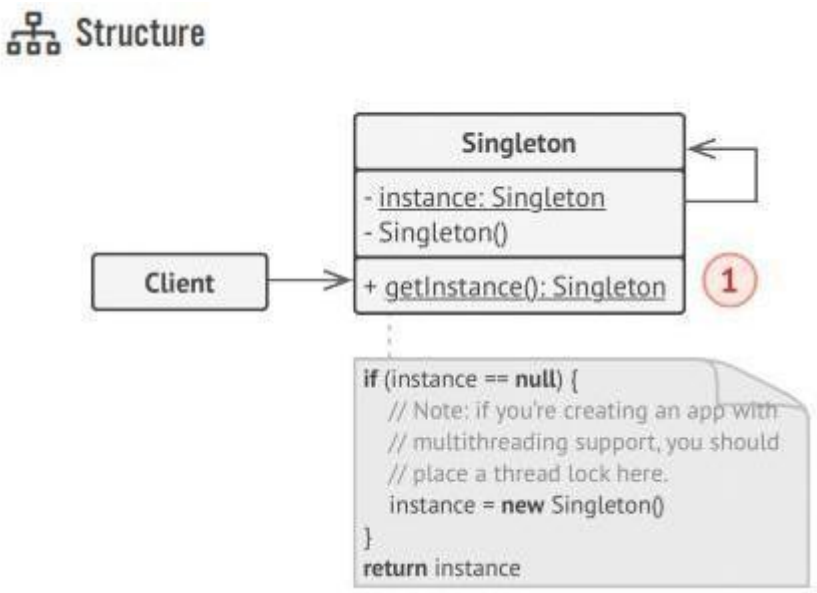
3. Behavioral Patterns

Deal with communication between objects. ***Example:*** Observer

In this lab, we study one from each category.

1. Singleton Design Pattern

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance



In many software systems, some resources must be shared across the entire application. Creating multiple objects for such resources can cause:

- Data inconsistency
- Resource conflicts
- Increased memory usage
- Synchronization problems

Singleton solves this by guaranteeing that only one object exists.

Real-World Scenario

Consider a **Database Connection Manager**.

In a large system:

- Many modules need database access
- Creating a new connection each time is costly
- Too many connections may overload the database

So, we maintain **one shared database connection**.

This is a perfect use case for Singleton.

How Singleton Works

1. Constructor is **private**
   → prevents new keyword use
2. Static instance variable stores object
3. Public static getInstance() method
   → controls object creation
4. First call creates object
   → later calls return same object

Code Example

```
class Database {

    private static Database instance;

    // Private constructor
    private Database() {
        System.out.println("Database connected");
    }

    // Thread-safe getInstance
    public static Database getInstance() {

        if(instance == null) {
            synchronized(Database.class) {
                if(instance == null) {
                    instance = new Database();
                }
            }
        }
        return instance;
    }

    // Business method
    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}
```

Client Code

```
class Application {
    public static void main(String[] args) {

        Database db1 = Database.getInstance();
        db1.query("SELECT * FROM users");

        Database db2 = Database.getInstance();
        db2.query("SELECT * FROM orders");

        // db1 and db2 refer to SAME object
    }
}
```

Explanation

1. Private Constructor

Prevents direct object creation.

2. Static Instance

Stores single object.

3. getInstance()

Creates object once, then reuses it.

4. Thread Lock

Prevents multiple threads from creating multiple instances.

Applicability

Use Singleton when:

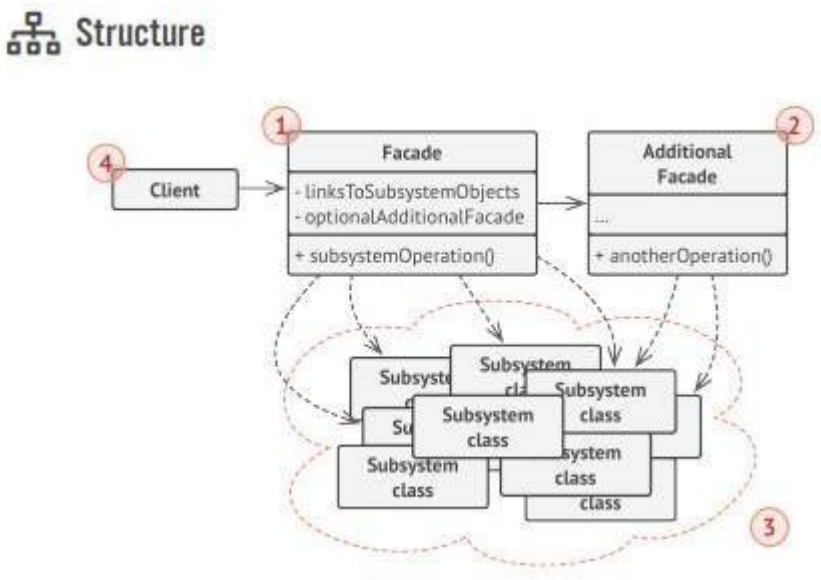
✔ Only one instance is needed
✔ Shared resource management required
✔ Strict control over global access needed

Examples:

- Database connection
- Logger system
- Configuration manager
- Cache manager

2. Facade Design Pattern

Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of Classes



The Facade pattern provides a **simplified interface to a complex subsystem**.

Large systems often consist of many classes that interact in complicated ways. Clients that use these systems must understand many internal details, which increases:

- Complexity
- Learning time
- Dependency on internal classes
- Maintenance difficulty

The Facade pattern hides this complexity by offering a single entry point.

Real-World Scenario

Consider a **video conversion system** using a third-party framework.

The framework contains many classes:

- VideoFile
- CodecFactory
- BitrateReader
- AudioMixer
- Multiple codecs

To convert a video, a developer must call these classes in the correct order.

This makes the code:

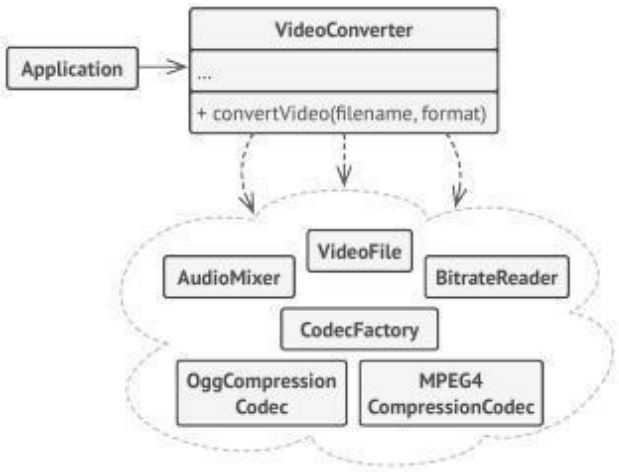
❌ Hard to use
❌ Hard to update
❌ Strongly coupled to framework

Facade solves this by wrapping everything into one class.

How Facade Works

1. A Facade class is created
2. It calls subsystem classes internally
3. Client interacts only with Facade
4. Subsystem complexity is hidden

Scenario based Diagram



An example of isolating multiple dependencies within a single facade class.

Code Example

```
class VideoFile {}
class OggCompressionCodec {}
class MPEG4CompressionCodec {}

class CodecFactory {
    static Object extract(VideoFile file) {
        return new OggCompressionCodec();
    }
}

class BitrateReader {
    static String read(String filename, Object codec) {
        return "buffer";
    }
```

```
class BitrateReader {
    static String read(String filename, Object codec) {
        return "buffer";
    }

    static String convert(String buffer, Object codec) {
        return "converted";
    }
}

class AudioMixer {
    String fix(String result) {
        return result + " fixed";
    }
}
```

```
// FACADE
class VideoConverter {
    public File convert(String filename, String format) {

        VideoFile file = new VideoFile();
        Object sourceCodec = CodecFactory.extract(file);

        Object destinationCodec;
        if(format.equals("mp4"))
            destinationCodec = new MPEG4CompressionCodec();
        else
            destinationCodec = new OggCompressionCodec();

        String buffer = BitrateReader.read(filename, sourceCodec);
        String result = BitrateReader.convert(buffer, destinationCodec);
        result = new AudioMixer().fix(result);

            return new File(result);
    }
}
```

Client Code

```
java

class Application {
    public static void main(String[] args) {
        VideoConverter converter = new VideoConverter();
        File mp4 = converter.convert("video.ogg","mp4");
    }
}
```

Explanation

✔ Client does not interact with codecs
✔ All complexity handled by Facade
✔ Framework can be replaced easily

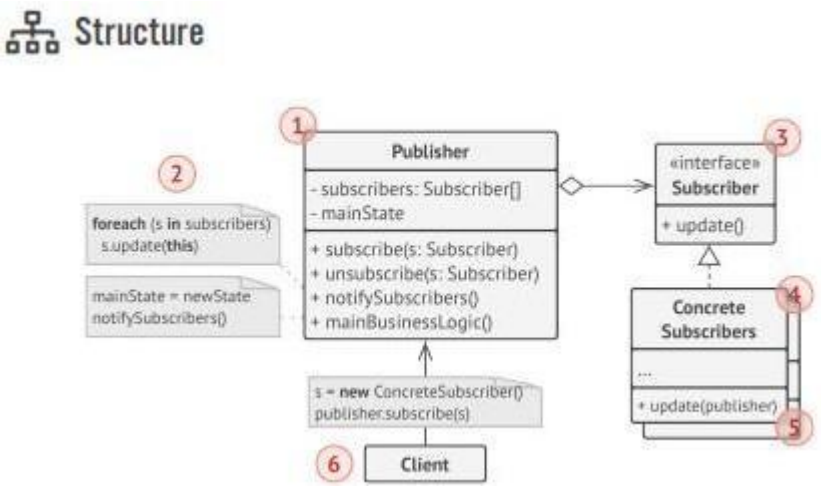
Applicability

Use Facade when:

✔ System is complex
✔ Many classes must be used together
✔ You want layered architecture
✔ You want to reduce coupling

3. Observer Pattern

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



The Observer pattern allows objects to be notified automatically when another object changes state.

It defines a **one-to-many relationship**:
One publisher → many subscribers

This is useful in event-driven systems.

Real-World Scenario

Consider a **text editor**.

When a file is:

- Opened
- Saved

The system must notify:

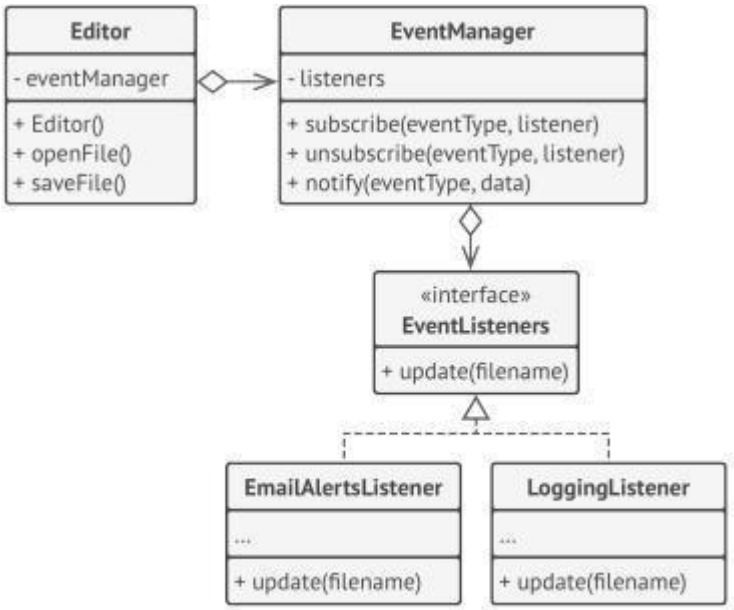
- Logging service
- Email alert system

Instead of hardcoding notifications, we use Observer.

How Observer Works

1. Publisher maintains subscriber list
2. Subscribers register/unregister
3. Publisher notifies on change
4. Subscribers react independently

Scenario based Diagram



Notifying objects about events that happen to other objects.

Code Example

```
interface EventListener {
    void update(String filename);
}

class EventManager {
    Map<String,List<EventListener>> listeners = new HashMap<>();

    void subscribe(String event, EventListener l){
        listeners.computeIfAbsent(event,k->new ArrayList<>()).add(l);
    }

    void notify(String event, String data){
        for(EventListener l:listeners.get(event))
            l.update(data);
    }
}

class Editor {
    EventManager events = new EventManager();
    String file;

    void openFile(String path){
        file = path;
        events.notify("open",file);
    }

    void saveFile(){
        events.notify("save",file);
    }
}

class LoggingListener implements EventListener{
    public void update(String filename){
        System.out.println("Log: "+filename);
    }
}

class EmailAlertsListener implements EventListener{
    public void update(String filename){
        System.out.println("Email alert: "+filename);
    }
}
```

Client Code

```java
class Application {
    public static void main(String[] args){

        Editor editor = new Editor();

        editor.events.subscribe("open",new LoggingListener());
        editor.events.subscribe("save",new EmailAlertsListener());

        editor.openFile("test.txt");
        editor.saveFile();
    }
}
```

Explanation

- ✔ The list of subscribers is compiled dynamically: objects can start or stop listening to notifications at runtime, depending on the desired behavior of your app.
- ✔ In this implementation, the editor class doesn't maintain the subscription list by itself. It delegates this job to the special helper object devoted to just that. You could upgrade that object to serve as a centralized event dispatcher, letting any object act as a publisher.
- ✔ Adding new subscribers to the program doesn't require changes to existing publisher classes, as long as they work with all subscribers through the same interface.

Applicability

Use Observer when:

- ✔ Event notification needed
- ✔ Many objects depend on one
- ✔ Loose coupling required
- ✔ Dynamic subscription needed

EXERCISE

Task 1 — Singleton Pattern

Scenario: University Examination System

A university is developing an **online examination system** used by thousands of students simultaneously.

The system includes:

- Student module
- Teacher module
- Result processing module
- Admin panel

All modules must access a **central Exam Configuration Manager** that stores:

- Exam duration
- Grading policy
- Passing criteria
- Exam rules

If multiple configuration objects are created:

- Rules may become inconsistent
- Different modules may follow different policies
- System integrity may break

Your Task

1. Explain why this scenario requires Singleton.
2. Design a Singleton class `ExamConfigManager`.
3. Ensure only one instance exists.
4. Show how different modules access the same instance.
5. Draw UML diagram.
6. Discuss what problem occurs if Singleton is not used.

Task 2 — Facade Pattern

Scenario: Smart City Service Portal

A city launches a **Smart City Portal** where citizens can request services like:

- Garbage collection
- Water supply complaint
- Streetlight repair
- Road maintenance

Each request requires interacting with multiple departments:

- Complaint Registration System
- Department Routing System
- SMS Notification System
- Tracking System
- Billing System

Currently, the client app directly interacts with all these subsystems, making the code complex.

Your Task

1. Identify problems in current system.
2. Design a `CityServiceFacade`.
3. Facade should provide simple methods like:

    - requestService()
    - trackRequest()

4. Show how facade hides complexity.
5. Draw UML diagram.
6. Explain how facade helps future

upgrades. **Task 3 — Observer Pattern**

Scenario: E-Learning Platform

An online learning platform wants to notify users when:

- A new lecture is uploaded
- Assignment deadline is near
- Grades are

released Subscribers

include:

- Students
- Teachers
- Academic advisors
- Parents (optional subscription)

Each subscriber wants different notifications.

Your Task

1. Identify publisher and observers.

2. Design an Observer system.

3. Allow dynamic subscription/unsubscription.

4. Implement at least two subscriber types.

5. Draw UML diagram.

6. Explain what happens if Observer is not used.

Task 4 — Combined Design Challenge

Scenario: Online Event Management Platform

An event platform manages:

- Event registration
- Notifications
- Payment processing
- Event rules

Requirements:

- Only one Payment Gateway Config (Singleton)
- Simplified booking interface (Facade)
- Notify users of event updates (Observer)

Your Task

1. Identify where each pattern applies.

2. Design system architecture.

3. Draw UML diagram.

4. Provide sample code for one pattern.

5. Justify design decisions.

Lab Session 08

Open Ended Lab

OBJECTIVE

The objective of this lab is to evaluate students' understanding and practical skills developed during the previous seven lab sessions. Students will apply concepts such as software documentation, UML modeling, design principles, design patterns, and user interface prototyping to analyze and design a small software system. The lab allows students to creatively apply the knowledge gained in earlier labs to a problem of their choice.

EVALUATION METHOD

Students will be evaluated through open-ended problem-solving tasks. Each student will select a **small software system of their choice** and perform basic design and documentation activities using the tools and techniques learned in previous lab sessions.

Evaluation will be carried out using the **OEL rubrics (5 marks)** where each criterion will be assessed as **Achieved (1)** or **Not Achieved (0)**.

TASKS

System Selection

Students will select any small software system of their choice with at least one user and a few basic features.

Task 1 – Software Description

Write a short description of the selected system using **LaTeX**. The description should include:

- System name
- Purpose of the system
- Main features of the system
- Types of users (if applicable)

Task 2 – UML Modeling

Create two UML diagram for the selected system using any diagramming tool. Students may choose any one of the following diagrams:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Collaboration Diagram

The diagram should clearly represent the structure or interaction of the system.

Task 3 – User Interface Prototype

Design a basic user interface prototype of the system using Figma. The prototype should include at least two screens, such as:

- Login or Home screen
- Main feature screen (e.g., task list, booking page, quiz page,

etc.) The design should follow basic **UI design principles**.

Task 4 – Design Concept Identification

Identify one design principle or design pattern (e.g., Singleton, Observer, or Facade) that could be applied in the selected system and briefly explain its possible use.

NED University of Engineering & Technology
Department of Software Engineering
OEL RUBRICS
Course Code and Title

Criteria	Achieved (1)	Not Achieved (0)	Marks
System description is clearly written using LaTeX including system purpose and features	Student successfully prepares the system description using LaTeX and includes key details such as purpose and features	Student does not prepare the description in LaTeX or the description is incomplete	1
UML diagram correctly represents the selected system	Student creates a correct UML diagram (Use Case / Class / Sequence) representing the system functionality	Diagram is missing, incorrect, or does not represent the system properly	1
User Interface prototype designed using Figma	Student designs at least two UI screens in Figma demonstrating the main functionality of the system	UI prototype is missing or fewer than two screens are designed	1
Identification of design principle or design pattern	Student correctly identifies and briefly explains a suitable design principle or design pattern for the system	Design principle or pattern is missing or incorrectly explained	1
Overall understanding and integration of concepts from previous labs	Student demonstrates clear understanding and integrates concepts learned in previous lab sessions	Student shows weak understanding or fails to integrate previous lab concepts	1
Weighted CLO Score			
Remarks and Signature			

Introduction

Lab Session 9 *Code Quality Analysis*

Code quality refers to the degree to which source code is written in a way that meets both functional and non-functional requirements, as well as best practices for readability, maintainability, efficiency, and reliability. High-quality code is:

1. **Readable:** Easy to understand for other developers, reducing time spent on code reviews and debugging.
2. **Maintainable:** Written in a way that changes can be easily made in the future without introducing bugs.
3. **Reliable:** It performs as expected with minimal risk of errors.
4. **Efficient:** It utilizes resources effectively, avoiding unnecessary computations.
5. **Consistent:** Adheres to established coding standards and guidelines.

Code Quality Tools

Several tools are available to help developers assess the quality of their code. Some common code quality tools include:

- **SonarQube**
- **ESLint** (for JavaScript/TypeScript)
- **Pylint** (for Python)
- **Checkstyle** (for Java)
- **Codacy**
- **PMD** (for Java)

These tools provide static analysis of the source code, detecting issues such as code smells,

potential bugs, security vulnerabilities, and deviations from coding standards.

Using SonarQube for Code Quality

In this lab, we will use **SonarQube** to check and improve code quality. It will help identify issues early, enforce coding standards, and ensure that our software is reliable and maintainable. By integrating SonarQube into our CI/CD pipeline, we can perform automated code quality checks every time code is committed or merged, maintaining high standards throughout the development lifecycle.

What is SonarQube?

SonarQube is an open-source platform that continuously inspects the quality of source code and provides comprehensive reports on its health. It helps teams write cleaner and safer code by identifying

1. **Bugs:** Issues in the code that may lead to incorrect behavior.
2. **Vulnerabilities:** Potential security risks in the source code.
3. **Code Smells:** Poor code practices that can lead to maintainability problems.
4. **Duplications:** Repeated code blocks that increase the maintenance burden.
5. **Test Coverage:** The extent of the code covered by automated tests.

SonarQube uses **static code analysis** to detect issues without running the code. It assigns severity levels to issues, ranging from minor to blocker, and generates a **Code Quality Report** with detailed insights, helping developers prioritize and fix problems early in the development process.

Getting Started with SonarQube Cloud

SonarQube Cloud, also known as **SonarCloud**, is a cloud-based code quality and security tool that integrates with popular DevOps platforms like GitHub, GitLab, Bitbucket, and Azure DevOps. It offers a comprehensive analysis of your projects, providing real-time feedback on code quality, helping you identify bugs, vulnerabilities, and code smells, and enforcing clean code standards.

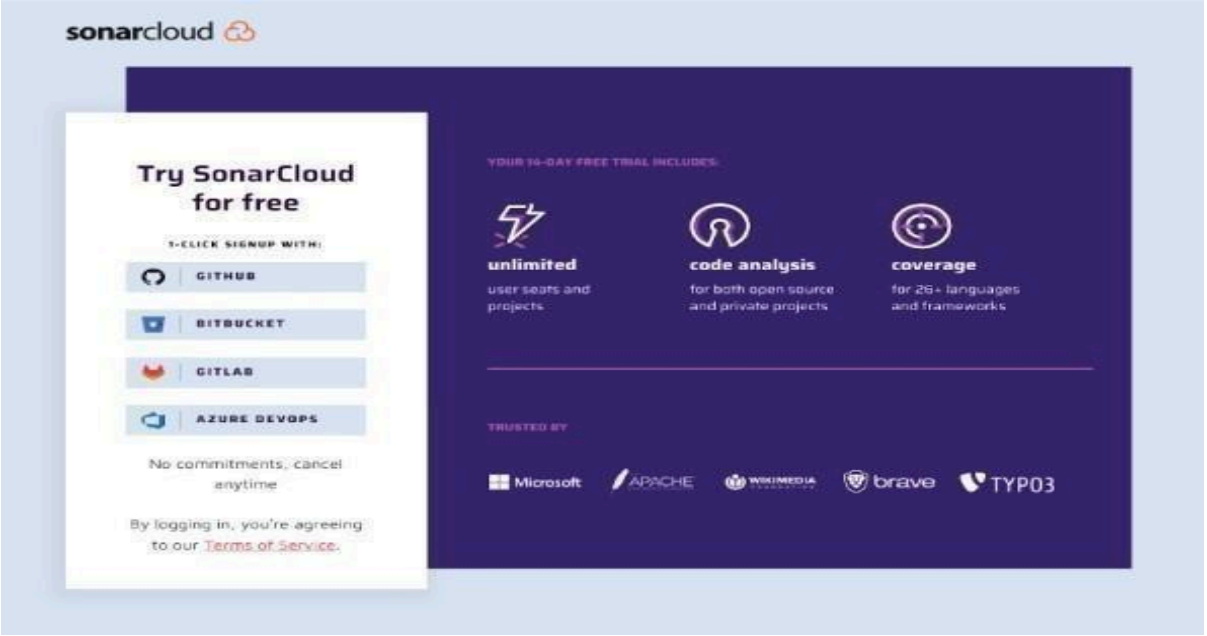
Here's a detailed step-by-step guide on how to get started:

Step 1: Sign Up and Integrate with Your Development Platform

To get started with SonarQube Cloud, visit the SonarCloud Sign Up page. Create an account using your preferred DevOps platform:

- **GitHub**
- **GitLab**
- **Bitbucket**
- **Azure DevOps**

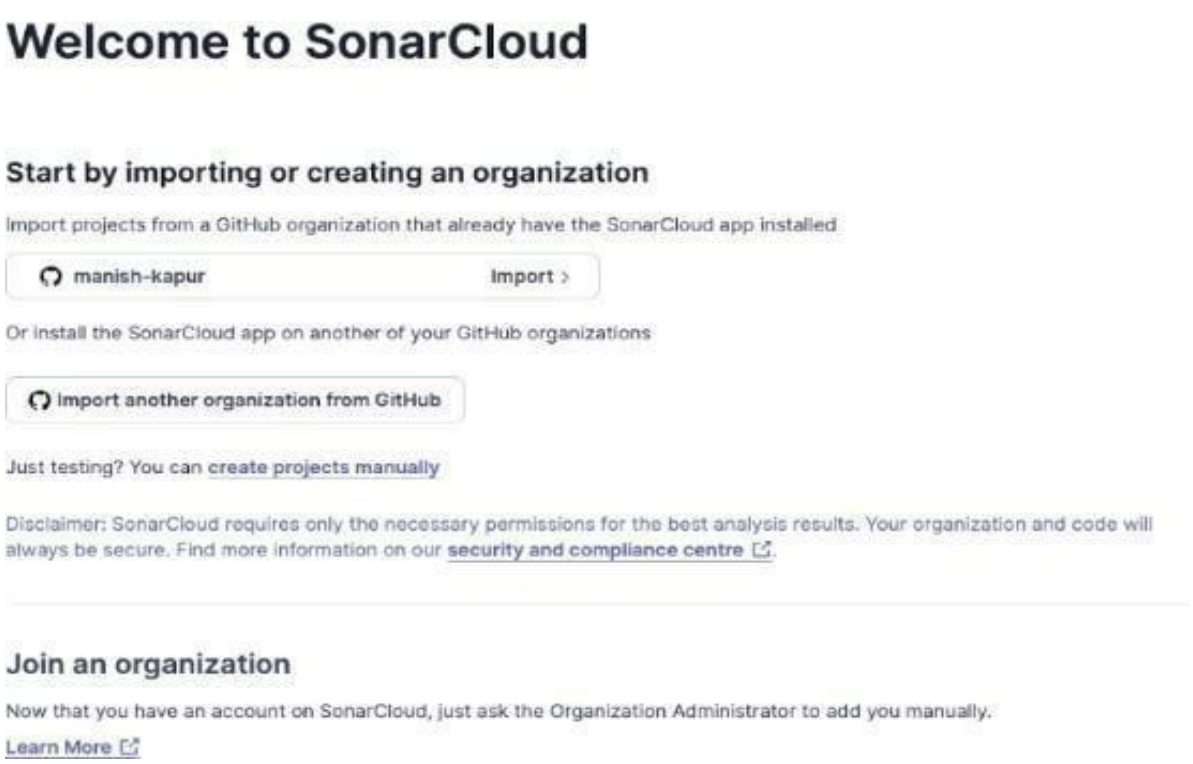
By connecting your DevOps account, SonarQube Cloud will link directly to your repositories, allowing seamless integration. You can choose your GitHub organization or personal account for setup



Step 2: Set Up Your Organization in SonarQube Cloud

Once you've connected your DevOps account, the next step is to create or select an organization in SonarQube Cloud:

- **Create a New Organization:** You can start fresh by setting up a new organization in SonarQube Cloud.
- **Import an Existing Organization:** If you already have an organization in your GitHub or GitLab account, you can import it directly. The imported organization will automatically include all members from your DevOps platform.

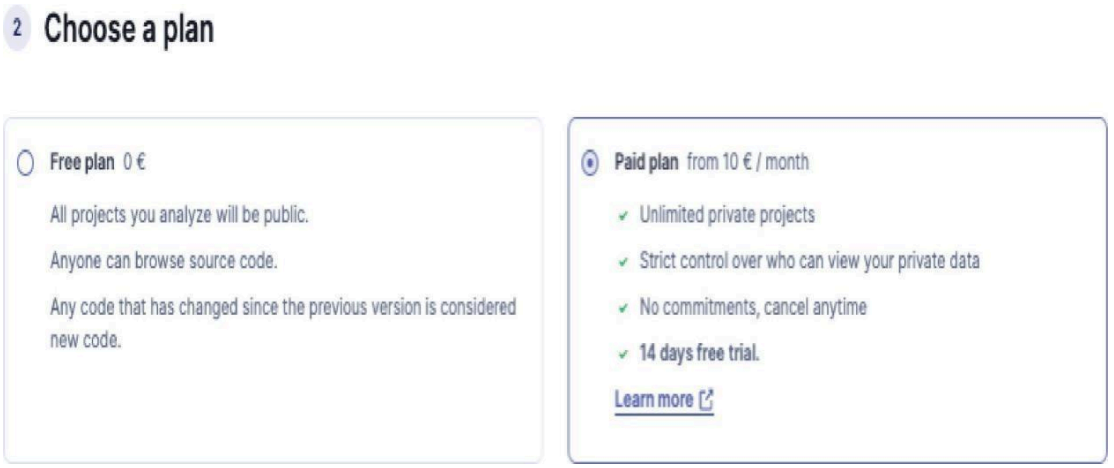


Step 3: Choose a Plan

SonarQube Cloud offers both free and paid plans:

- **Free Plan:** This is suitable for public repositories and includes basic code analysis features.
- **Paid Plan (14-Day Trial):** This plan is required for analyzing private repositories. You can start a no-commitment 14-day free trial. A credit card is required for the trial, but you won't be charged until the trial ends. Pricing is based on the number of Lines of Code (LOC) analyzed.

During the trial, you get access to all features, including advanced code analysis, Quality Gates, and detailed reporting.



Step 4: Import and Analyze Your First Repository

Once your organization is set up, it's time to import your projects:

1. Navigate to your organization's **Projects** tab in SonarQube Cloud.
2. Select the repositories you want to analyze from your GitHub, GitLab, Bitbucket, or Azure DevOps account.
3. Click **Set Up** to start the configuration.

Step 5: Run Your First Code Analysis

SonarQube Cloud offers two methods for running code analysis:

- **Automatic Analysis:** This method triggers analysis automatically when code is pushed to your repository.
- **CI-Based Analysis:** For greater control, you can integrate SonarQube Cloud with your CI/CD tool (e.g., GitHub Actions, GitLab CI/CD). This method allows you to run analysis during your build process using sonar-scanner.

Example GitHub Actions workflow: name:

```
Code Quality Analysis
on: [push,
pull_request] jobs:
sonarQubeScan:
  runs-on:
  ubuntu-latest steps:
    - uses: actions/checkout@v3
    - name:
      Run
      SonarScanner
      run:
```

sonar-scanner

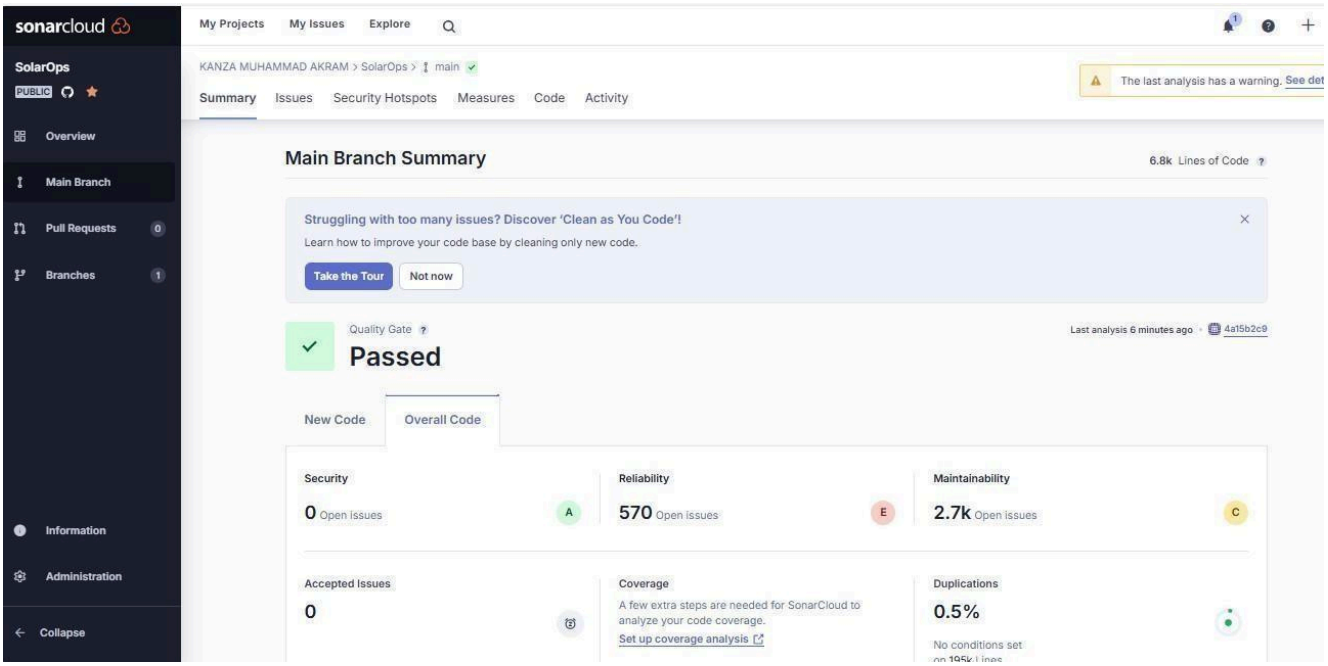
This setup ensures that every new pull request and push is analyzed automatically, providing immediate feedback.

Step 6: Explore the SonarQube Cloud Dashboard

After the first analysis is complete, visit the **SonarQube Cloud dashboard** to review the results. Here's what you can find:

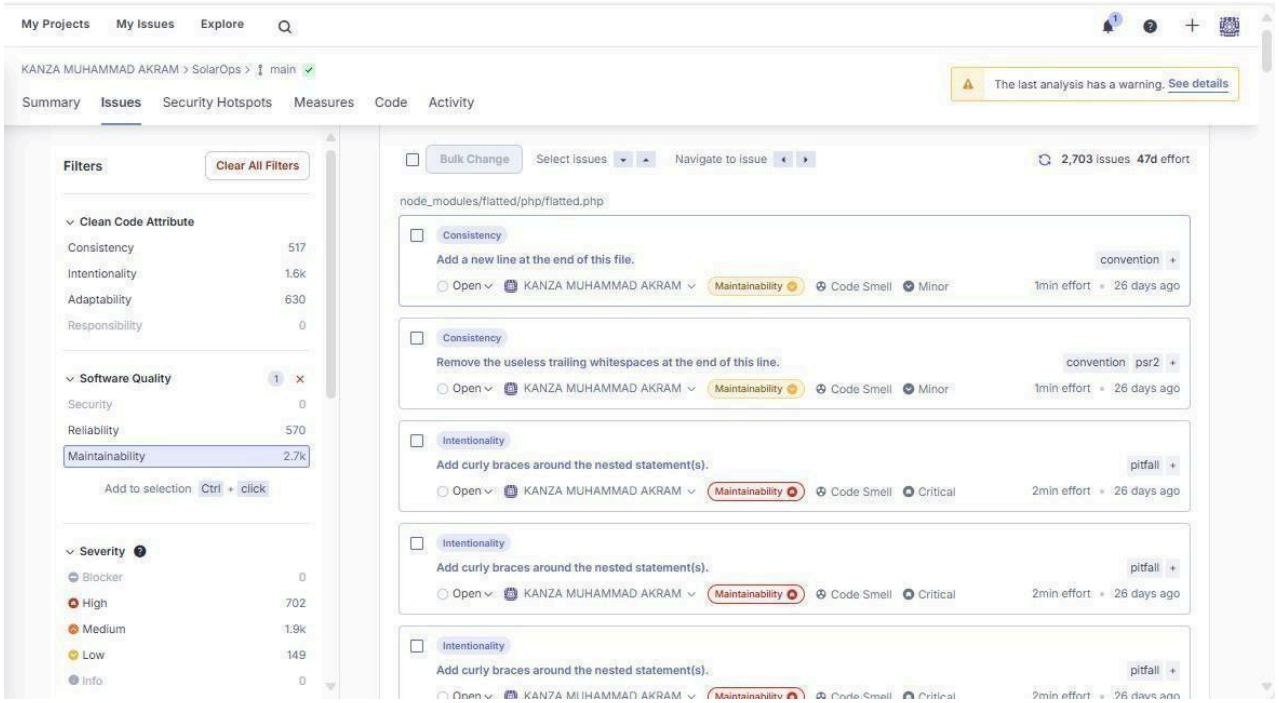
- **Bugs, Vulnerabilities, Code Smells:** See a list of issues detected in your code, along with recommendations for fixing them.
- **Code Coverage:** View the percentage of your code that is covered by tests, helping you identify areas that need more testing.
- **Quality Gate Status:** Check if your project passes the defined Quality Gate criteria, which include thresholds for code coverage, bugs, and vulnerabilities.

Spend time familiarizing yourself with these metrics to prioritize your code quality improvements effectively.



Step 7: Dive Into the Issues and PR Analysis

- When you create a pull request, SonarQube Cloud analyzes the changes and provides feedback right in the PR interface.
- Issues detected in the pull request are displayed, and the quality gate status is shown. If the code does not meet the defined criteria, the pull request can be blocked from merging.


My Projects
My Issues
Explore
KANZA MUHAMMAD AKRAM > SolarOps > main
Summary
Issues
Security Hotspots
Measures
Code
Activity
The last analysis has a warning. See details
Filters
Clear All Filters
Clean Code Attribute
Consistency 517
Intentionality 1.6k
Adaptability 630
Responsibility 0
Software Quality 1
Security 0
Reliability 570
Maintainability 2.7k
Add to selection Ctrl + click
Severity
Blocker 0
High 702
Medium 1.9k
Low 149
Info 0
Bulk Change
Select issues
Navigate to issue
2,703 issues 47d effort
node_modules/flatted/php/flatted.php
Consistency
Add a new line at the end of this file.
convention
Open
KANZA MUHAMMAD AKRAM
Maintainability
Code Smell
Minor
1min effort • 26 days ago
Consistency
Remove the useless trailing whitespaces at the end of this line.
convention psr2
Open
KANZA MUHAMMAD AKRAM
Maintainability
Code Smell
Minor
1min effort • 26 days ago
Intentionality
Add curly braces around the nested statement(s).
pitfall
Open
KANZA MUHAMMAD AKRAM
Maintainability
Code Smell
Critical
2min effort • 26 days ago
Intentionality
Add curly braces around the nested statement(s).
pitfall
Open
KANZA MUHAMMAD AKRAM
Maintainability
Code Smell
Critical
2min effort • 26 days ago
Intentionality
Add curly braces around the nested statement(s).
pitfall

EXERCISE

Analyze any existing GitHub project using SonarQube Cloud. Perform a full code quality analysis and capture screenshots of your findings, including code smells, bugs, vulnerabilities, maintainability, reliability graphs, and coverage. Document all your observations, summarize the issues, and suggest improvements in a brief report.

Lab Session 10

Software Testing - Manual Testing

SOFTWARE TESTING

Software testing is an important activity in the **software development lifecycle (SDLC)** that ensures the software application works correctly and meets the specified requirements. The purpose of software testing is to identify defects, verify functionality, improve reliability, and ensure that the system behaves as expected under different conditions.

Testing plays a critical role in delivering **high-quality software products**. Without proper testing, software systems may contain bugs that could cause system failures, incorrect outputs, security vulnerabilities, or poor user experience.

Software testing helps to:

- Detect defects before deployment
- Verify that requirements are correctly implemented
- Improve software reliability and stability
- Enhance system performance
- Ensure a better user experience

Software testing can be broadly categorized into two major approaches:

- **Manual Testing**
- **Automation Testing**

Both approaches are widely used in software development depending on project requirements.

AUTOMATION TESTING

Automation testing uses specialized tools and scripts to automatically execute test cases. Instead of manually performing each test step, testers create automated scripts that simulate user actions and verify expected outcomes.

Automation testing is particularly useful for **repetitive and large-scale testing tasks**. It is commonly used for:

- Regression testing
- Performance testing
- Load testing
- Continuous integration testing

Automation tools commonly used in the industry include:

- Selenium
- Cypress
- TestComplete
- Appium

Although automation testing provides faster execution and improved efficiency, it requires initial setup, programming knowledge, and maintenance of test scripts.

MANUAL TESTING

Manual testing is the process of testing software **manually without using automation tools**. In this method, testers interact with the application just like real users and verify whether the system behaves according to the expected requirements.

Manual testing allows testers to explore the application and detect issues related to usability, interface design, and unexpected behaviors that automated tests might miss.

Manual testing is commonly used for:

- Exploratory testing
- Usability testing
- Interface testing
- Ad-hoc testing

Even in projects where automation testing is implemented, manual testing remains essential for validating **user experience and complex user scenarios**.

STEPS FOR MANUAL TESTING

Manual testing generally follows a structured process to ensure systematic verification of software functionality.

1. Define the Test Scenario

A test scenario describes a **high-level functionality** that needs to be tested.

Example:

Test Scenario:
Verify the login functionality of the **Facebook login page**.

This scenario defines the feature being tested without going into detailed steps.

2. Create Test Case ID

Each test case must have a **unique identifier** for tracking and documentation purposes.

Example:

- TC-001
- TC-002
- TC-003

Unique IDs help testers easily reference and manage test cases.

3. Test Case Description

The description explains **what the test case verifies**.

Example:

Verify that a user can successfully log in using valid credentials.

4. Preconditions

Preconditions are the conditions that must be satisfied before executing the test.

Examples:

- User must have a registered Facebook account
- Internet connection must be active
- User must be on the login page

5. Test Steps

Test steps define the **exact sequence of actions** the tester must perform.

Example steps:

1. Open Facebook login page
2. Enter valid username
3. Enter valid password
4. Click the **Login** button

6. Expected Result

The expected result describes what the system **should do** if the test passes.

Example:

User should be successfully logged in and redirected to the homepage.

7. Actual Result

The actual result is the **observed system behavior** after executing the test.

Example:

User logged in successfully and homepage displayed.

8. Status

Based on the comparison of **expected vs actual result**, the test case is marked as:

- **Pass**
- **Fail**

9. Remarks

Remarks include additional comments or observations made during testing.

Example:

Login fails when password contains special characters.

TEST SCENARIO

Scenario: Verify the Login Functionality of Facebook Login Page

The login page allows users to access their accounts by entering valid credentials. The following functionalities should be verified:

- Login with valid credentials
- Login with incorrect password
- Login with invalid username
- Login with empty fields

TEST CASE DOCUMENTATION FORMAT

Before writing test cases, detailed information about the test should be recorded.

Test Case Details

Test Case Name: Login
Created By: _____
Executed By: _____
Date Created: _____
Execution Date: _____
Application Name: Facebook

Test Case Table

Test ID	Test Name	Test Description	Precondition	Test Steps	Expected Result	Actual Result	Post Condition	Status
TC-01	Login with Valid Credentials	Verify that user can login with valid username and password	User must have a registered Facebook account	**1.** Open Facebook login page **2.** Enter valid username **3.** Enter valid password 4. Click Login button	User should be redirected to homepage after successful login	User redirected to homepage	User session starts	**Pass**
TC-02	Login with Invalid Password	Verify system behavior when user enters incorrect password	User must have a valid account	1. Open login page 2. Enter valid username 3. Enter incorrect password 4. Click Login	System should display error message "Incorrect password"	"Incorrect password"	User remains on login page	**Pass**
TC-03	Login with Invalid Username	Verify behavior when username does not exist	Internet connection active	1. Open login page 2. Enter invalid username 3. Enter password 4. Click Login	System should display error message "User not found"	User not found	Login denied	**Pass**

TC-04	Login with Empty Fields	Verify that system prevents login when fields are empty	User on login page	1. Open login page 2. Leave username and password empty 3. Click Login	Validation message should appear	Validation message appear	Login prevented	**Pass**

TC-05	Login with Username Only	Verify behavior when password field is empty	User on login page	1. Enter username 2. Leave password empty 3. Click Login	System should show validation error	Validation error	Login denied	**Pass**
TC-06	Login with Password Only	Verify behavior when username field is empty	User on login page	1. Leave username empty 2. Enter password 3. Click Login	Validation message displayed	Validation message	Login denied	**Pass**

EXERCISE

Task 1

Write code and prepare test cases for a function that **returns the largest of three real numbers**.

The function accepts three numbers as input and returns the largest value among them.

Students should test scenarios such as all three numbers are different, two numbers are equal,Negative numbers etc

Prepare **at least 6 test cases** that verify the correctness of the function.

Task 2

Write code and Design test cases for a function that **calculates the Least Common Multiple (LCM) of two positive integers**.

The function takes two positive integers and returns their least common multiple.

Students should consider cases such as:

- Two different numbers
- Two identical numbers
- Prime numbers
- One number larger than the other
- Edge cases with small values

Prepare **at least 6 test cases** to verify the correctness of the function.

Task 3

Design test cases for a function that **calculates the square root of a real number**.

The function receives a real number as input and returns its square root.

Students should analyze different situations such as Positive numbers, Invalid inputs etc

Prepare **at least 6 test cases** for this function.

Task 4

Write code and prepare test cases for a function that **sorts an array of integers in ascending order**.

The function receives:

- An array **A**
- A variable **n**, which represents the number of elements in the array

The function returns the array sorted in ascending order.

Prepare **at least 6 test cases** verifying that the sorting operation works correctly.

Lab Session 11

Agile Sprint Planning Using Trello

1. Introduction

In modern software development, Agile methodology is widely used to manage projects that require flexibility, rapid delivery, and collaboration. Unlike traditional waterfall models, Agile emphasizes iterative development, frequent feedback, and adaptive planning.

The Scrum framework is one of the most popular Agile frameworks. It organizes work into time-boxed iterations called sprints, where the team delivers working software incrementally.

The goal of this lab is to teach students how to plan and manage a sprint using Scrum principles, visualize tasks using Trello, and monitor progress using priority and burn-down charts.

2. Learning Objectives

By completing this lab, students will learn to:

1. Understand **Agile methodology and Scrum framework**
2. Define **Sprint Goals** aligned with project objectives
3. Select **user stories** from a product backlog and prioritize them
4. Break user stories into actionable tasks
5. Assign tasks to team members effectively
6. Estimate effort using hours or story points
7. Set up and manage a Trello Sprint Board
8. Draw priority charts and burn-down charts
9. Conduct a reflection on sprint planning, task management, and coordination

3. Agile Methodology

Agile methodology is a **set of principles for software development** that prioritize:

- **Individuals and interactions** over rigid processes and tools
- **Working software** over extensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a strict plan

Key Benefits of Agile:

- Faster delivery of functional software
- Improved customer satisfaction through frequent feedback
- Adaptable to changing requirements
- Increased team collaboration and accountability

Agile Practices:

1. **Iterative Development:** Break the project into small increments delivered in sprints.
2. **Continuous Feedback:** Engage stakeholders regularly to validate functionality.
3. **Adaptive Planning:** Reassess priorities after each iteration.
4. **Collaborative Teams:** Cross-functional teams work together on design, development, and testing.

4. Scrum Framework

Scrum is a lightweight framework for managing complex projects using Agile principles.

4.1 Scrum Roles

Role	Responsibilities
Product Owner	Manages the product backlog, prioritizes stories, ensures team builds the right features
Scrum Master	Facilitates Scrum processes, removes obstacles, ensures team follows Agile practices
Development Team	Implements and tests tasks, self-organizing, responsible for delivering potentially shippable increments

4.2 Scrum Artifacts

1. **Product Backlog** – A complete list of features, enhancements, and bug fixes needed for the product.
2. **Sprint Backlog** – Subset of the product backlog selected for a sprint.
3. **Increment** – Completed, functional product delivered at the end of a sprint.

4.3 Scrum Ceremonies

1. **Sprint Planning:** Define sprint goal, select stories, break them into tasks, assign work.
2. **Daily Stand-up:** Quick daily meeting to review progress and obstacles.
3. **Sprint Review:** Demonstrate the completed increment to stakeholders.
4. **Sprint Retrospective:** Reflect on the sprint to improve team processes.

5. Sprint Planning

Sprint Planning ensures that the development team knows:

- **What to build** (Sprint Goal)
- **How to build it** (Tasks, assignments, and dependencies)
- **How long it will take** (Effort estimation)

5.1 Steps in Sprint Planning

1. **Define Sprint Goal** – Clear objective for the sprint, aligned with product vision.
2. **Select User Stories** – Choose items from the product backlog achievable within the sprint.
3. **Prioritize Stories** – Identify high-value stories first.
4. **Break Stories into Tasks** – Smaller tasks are easier to estimate and assign.
5. **Estimate Effort** – Use hours or story points for planning capacity.
6. **Assign Tasks** – Allocate work based on team members' expertise.
7. **Plan Sprint Board** – Visualize tasks in **To Do, In Progress, Done** columns.
8. **Visualize Progress** – Priority charts and burn-down charts track performance.

6. Product Backlog

Before we can begin a sprint, we need to **prepare a product backlog**.

What is a Product Backlog?

- A product backlog is a **living document** listing all features, enhancements, bug fixes, and tasks that could be done for a project.
- Each item is written as a **user story**, e.g., "As a user, I want to log in so that I can access my account."
- Stories should be **prioritized** by value, risk, and dependencies.

Breaking Stories into Tasks:

- Each story should be broken into **smaller, manageable tasks**.
- Tasks are easier to **estimate, assign, and track** than full user stories.
- Example tasks for a "Login" story:
    - Design login form UI
    - Implement backend authentication
    - Connect UI with backend
    - Test login functionality

Example Product Backlog Table

Students create the **Product Backlog Table** in Word.

ID	User Story	Description	Priority
US1	User Login	Implement login functionality	High
US2	User Registration	Sign-up form & validation	High
US3	Profile Management	Edit profile details	Medium
US4	Dashboard	Display recent activity	Medium
US5	Notifications	Alerts for updates	Low

7. Breaking Stories into Tasks

- Breaking stories into tasks ensures clarity, accurate estimation, and easier tracking.
- Each task should ideally be **completable within 1–2 days**.

Example: US1 – User Login

Task ID	Task Description	Assigned To	Estimated Hours	Status
T1.1	Design login form UI	Front-end	3	To Do
T1.2	Implement backend authentication	Back-end	4	To Do
T1.3	Connect UI with backend	Front-end	2	To Do
T1.4	Test login functionality	QA	2	To Do

8. Priority Chart

- Priority charts **visualize which stories/tasks are most important**.
- Helps the team focus on **high-value features first**.

Students count tasks according to priority.

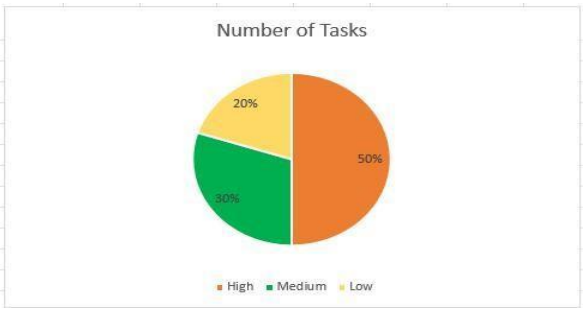
Steps

1. Open **Microsoft Excel**
2. Enter the table
3. Select the table
4. Click **Insert**
5. Choose **Bar Chart or Pie Chart**

Format:

X-axis → Priority
Y-axis → Number of Tasks

Priority	Number of Tasks
High	5
Medium	3
Low	2



9. Trello Sprint Board

- Trello is a **Kanban-style board** to manage tasks.
- Columns (lists) represent **workflow stages**.

Students manage task using trello.

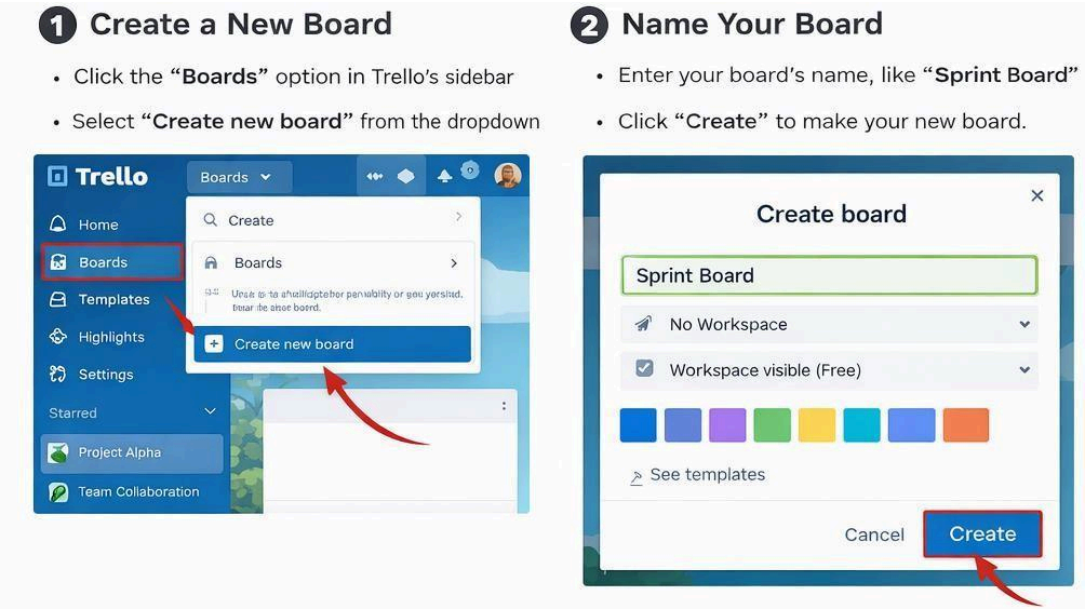
Step-by-step Instructions:

1. Create a new Trello board.
2. Create **columns**: To Do | In Progress | Done
3. Add **cards** for each task.
4. Assign **team members** to each card.
5. Move cards from **To Do → In Progress → Done** as work progresses.

Example Layout:

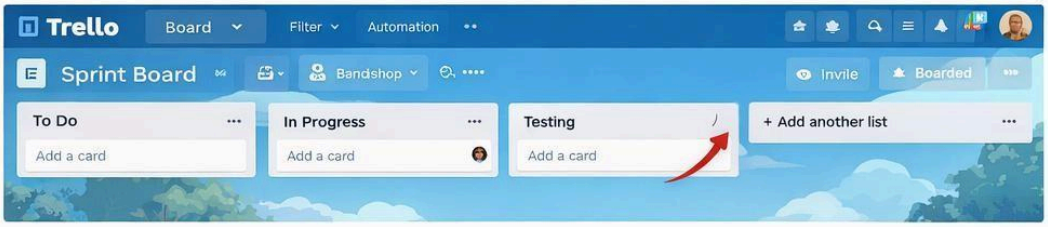
- **To Do:** T1.1, T1.2, T2.1
- **In Progress:** T1.3
- **Done:** T1.4

Tip: Use **labels** to show priority, story type, or department.



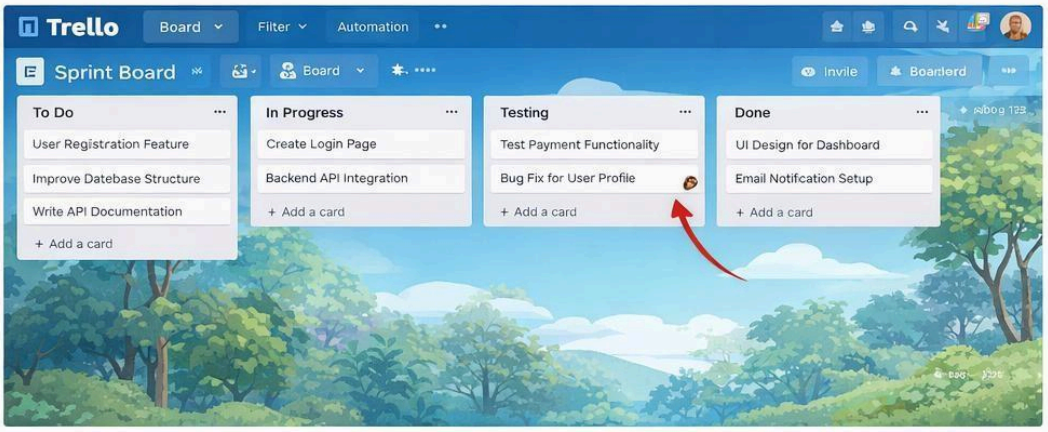
3 Add Lists to the Board

- Create lists like “To Do”, “In Progress,” “Testing,” and “Done”
- Click “Add a list” button to add each list.



4 Add Tasks and Start Sprint

- Add tasks to the “To Do” list for your sprint.
- Move tasks through the lists as they progress.



10. Burn-down Chart

- Burn-down charts track **remaining work vs sprint days**.
- Helps monitor progress and identify if the team is on schedule.

Students track **remaining work each day**.

Step-by-step Instructions:

1. Record **estimated hours** for each task in Trello (custom field or card description).
2. At the end of each day, calculate **remaining hours**.
3. Create a table in Excel or Sheets:

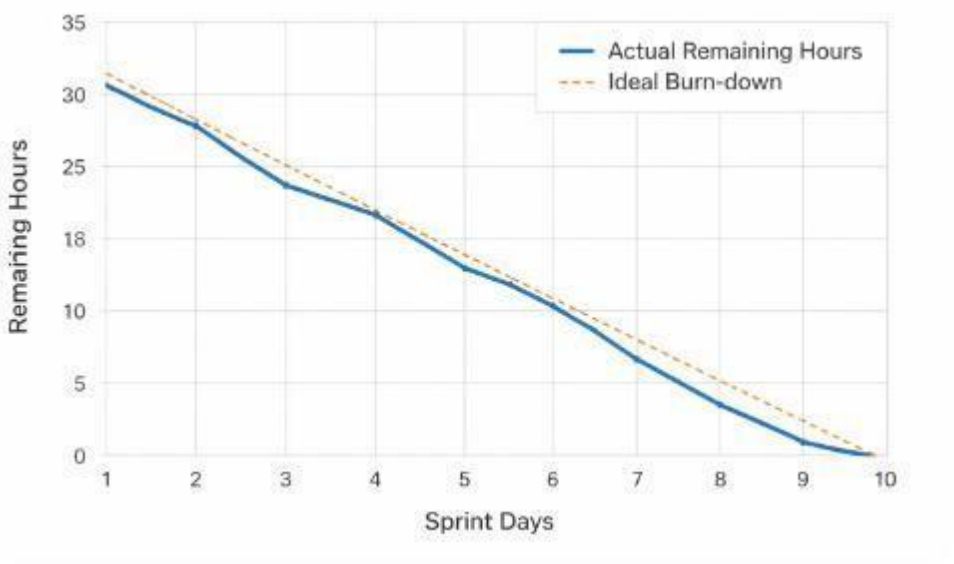
Day	Remaining Hours
1	32
2	28
3	25
4	20
5	16
6	12

7	9
8	5
9	2
10	0

4. Insert a **line chart**:

   - X-axis = Sprint Days
   - Y-axis = Remaining Hours
   - Plot actual progress against ideal trend line

Optional: Use Trello Power-Ups like Corrello for automated burn-down charts.



EXERCISE

Project: Simple Calculator Web Application

Instructions:

1. Define **Sprint Goal**
2. Prepare **Product Backlog**: Addition, Subtraction, Multiplication, Division, Input Validation, Error Handling, Responsive UI
3. Break each story into tasks, estimate effort, assign team members
4. Create **Trello Sprint Board**
5. Draw **priority chart** and **burn-down chart**

Deliverables:

- Sprint Goal document
- Sprint Backlog table with tasks, assignments, estimated hours
- Trello board screenshot
- Priority and burn-down charts
- Reflection paragraph

Lab Session 12

Introduction to GitHub: Repository Management and Version Control

1. Introduction

In modern software development, **version control** is a critical skill. Version control systems allow developers to **track changes, collaborate, and maintain the history of code**. **Git** is the most widely used version control system, and **GitHub** is a cloud-based platform that hosts Git repositories, providing collaboration tools, issue tracking, and online storage.

This lab introduces students to **GitHub fundamentals** and basic Git commands. Students will create repositories, manage files, commit changes, push to GitHub, and understand the workflow of version control.

2. Objectives

After completing this lab, students will be able to:

1. Understand the role of Git and GitHub in software development.
2. Create a new GitHub repository.
3. Clone a repository to a local machine.
4. Stage, commit, and push changes to GitHub.
5. Pull updates from a remote repository.
6. Understand branching and basic collaboration concepts.

3. Theory

3.1 Git and GitHub

- **Git** is a distributed version control system that tracks changes in files.
- **GitHub** is a web-based platform that hosts Git repositories, enabling collaboration.
- **Local repository**: The copy of a repository on your computer.
- **Remote repository**: The repository hosted online (GitHub).

3.2 Basic Git Workflow

1. **Clone a repository** → get a local copy.
2. **Edit files locally** → make changes.
3. **Stage changes** → mark changes to be committed.
4. **Commit changes** → save changes to local Git history.
5. **Push changes** → update the remote repository.
6. **Pull changes** → fetch updates from remote to local.

4. Tools and Resources

- **Git** (install if not already: https://git-scm.com/)
- **GitHub account** (https://github.com/)
- **Text editor** (VS Code recommended)
- Terminal/Command Prompt

5. Lab Procedure

Step 1: Set Up Git on Your Machine

1. ***Check if Git is installed:***

```bash
git --version
```

- Output example: git version 2.42.0
- If Git is not installed, download and install from https://git-scm.com/downloads

2. ***Configure Git with your username and email:***

```bash
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

Explanation: Git uses this information to track commits.

Step 2: Create a GitHub Repository

1. Log in to GitHub.
2. Click **"New Repository"**.
3. Enter repository name, e.g., MyFirstRepo.
4. Select **Public** or **Private**.
5. Initialize with a **README** (optional).
6. Click **"Create Repository"**.

Explanation:

- A repository is a storage space for your project.
- Public repositories can be seen by anyone; private are restricted.

Step 3: Clone the Repository Locally

1. Copy the repository URL from GitHub (HTTPS recommended).
2. Open terminal/command prompt.
3. Run:

```bash
git clone https://github.com/username/MyFirstRepo.git
```

Explanation:

- Cloning creates a local copy of the repository.
- Students can now edit files locally.

Step 4: Add a File and Stage Changes

1. Navigate into the repository folder:

```bash
cd MyFirstRepo
```

2. Create a file, e.g., index.html.
3. Add some content:

```html
<!DOCTYPE html>
<html>
<head>
    <title>My First Repo</title>
</head>
<body>
    <h1>Hello, GitHub!</h1>
</body>
</html>
```

4. Stage the file:

```bash
git add index.html
```

Explanation:

- git add tells Git to include this file in the next commit.

Step 5: Commit Changes

```
Bash

git commit -m "Add index.html with basic HTML structure"
```

Explanation:

- git commit records changes in the local repository.
- -m allows adding a commit message describing the changes.

Step 6: Push Changes to GitHub

```
Bash

git push origin main
```

Explanation:

- git push uploads local commits to the remote repository.
- origin refers to the remote repository.
- main is the default branch name (used to be master).

Step 7: Pull Updates from Remote Repository

```
Bash

git pull origin main
```

Explanation:

- git pull fetches updates from the remote repository and merges them into the local repository.
- Important when multiple people work on the same project.

Step 8: Branching (Optional Advanced Step)

1. Create a new branch:

```
Bash

git checkout -b feature-branch
```

2. Make changes, stage, and commit.
3. Push the branch:

```bash
git push origin feature-branch
```

4. On GitHub, create a pull request to merge the branch into main.

Explanation:

- Branches allow developers to work on new features without affecting the main code.
- Pull requests are used to review and merge changes.

EXERCISE

1. Create a GitHub repository and clone it to your machine.
2. Add a new file, stage it, commit it, and push it to GitHub.
3. Modify the file and push the update. Observe changes on GitHub.
4. Pull updates from remote if changes are made online.
5. Optional: Create a new branch, make changes, push it, and open a pull request.
6. Explain the purpose of each Git command used in the lab.

Lab Session 13

Complex Engineering Activity

Program Learning Outcome Mapping

PLO5 – Modern Tool Usage

An ability to create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, to complex engineering problems, with an understanding of the limitations. **(P3- PLO5)**

Complex Problem Solving Attributes (CPA)

CPA-1 (Depth of Analysis Required)

The activity requires students to analyze a software system and apply multiple software engineering tools across different phases of the Software Development Life Cycle (SDLC). Students must interpret outputs generated by tools such as project planning software, modeling tools, development and deployment tools.

CPA-2 (Familiarity of the Problem)

The problem is based on real-world software systems commonly used in everyday applications such as booking systems, management systems, and online services. Students must analyze system requirements and design solutions using appropriate engineering tools.

CPA-3 (Integration of Multiple Tools)

Students must integrate several modern software engineering tools including documentation tools, planning tools, modeling tools, code quality analysis tools, testing techniques and version control platforms to solve the problem effectively.

Problem Statement

Modern software engineering requires developers to build complete software systems using multiple tools for planning, design, development, testing, and deployment.

In this activity, students will select a small software system of their choice, develop a working system, and apply modern software engineering tools for documentation, project planning, UML modeling, quality analysis, testing, and deployment. The goal is to demonstrate the ability to develop a functional software system using modern engineering tools, while understanding their purpose, limitations, and integration.

Students may select **any small software system** such as:

- Student Task Management System
- Online Quiz System
- Event Registration System
- Library Management System
- Appointment Booking System
- Personal Expense Tracker
- Online Course Registration System

Students may also propose another similar system.

Guidelines and Evaluation Criteria Overview

Students must perform the following activities using the tools practiced during previous lab sessions.

Task 1 – System Documentation

Prepare a brief system description using LaTeX including:

- System overview
- System users
- Key features of the system
- Functional requirements

Task 2 – Project Planning

Create a project schedule using Microsoft Project including:

- Major development tasks
- Task dependencies
- Estimated duration of tasks

Task 3 – System Modeling

Create UML diagrams representing the system.

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- State Chart Diagram

Task 4 – System Development

Develop a **working software system** implementing the core features of the selected system.

Task 5 – Code Quality Analysis

- Perform **s**tatic code analysis using SonarQube Cloud.
- Students must analyze code quality metrics such as bugs, vulnerabilities, and code smells.

Task 6 – Manual Software Testing

Design test cases for the selected system including:

- Test case description
- Test steps
- Expected result
- Actual result

Task 7 – Version Control and Deployment

Upload project artifacts to GitHub and deploy a simple webpage or documentation site using GitHub Pages.

Students must provide:

- GitHub repository link
- GitHub Pages deployment link

Evaluation Criteria

Component	Taxonomy Level	Marks
Complete system development and design on selected tools (LaTeX, Microsoft Project, UML, Figma, Testing, GitHub)	P3	10
Total		**10**

Submission Details

Each student must submit the following:

1. LaTeX documentation (PDF format)
2. Microsoft Project planning screenshot
3. UML diagrams
4. Working system code
5. Sonar Qube Analysis Screenshots
6. Manual testing report with test cases
7. GitHub repository link
8. GitHub Pages deployment link

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
Course Code and Title

Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
Planning & Documentation (LaTeX + Project Schedule)	Clear documentation prepared in LaTeX and complete project plan created in MS Project with appropriate tasks and schedule	Documentation and planning partially complete or limited clarity	Documentation and planning missing or unclear	CPA-1	2
UML System Design	Appropriate UML diagrams created and correctly represent system structure and behavior	Diagrams partially correct or incomplete	UML diagrams missing or incorrect	CPA-2	2
Working System Development	System is fully functional and implements all core requirements	System partially working with limited functionality	System missing or non-functional	CPA-3	2
Code Quality Analysis & Testing (SonarQube + Manual Testing)	SonarQube analysis performed and manual test cases executed with proper results	Analysis or testing partially performed	Code analysis and testing not performed	CPA-3	2
Version Control & Deployment (GitHub/GitHub Pages)	Repository complete, system deployed successfully, instructions clear	Repository exists but deployment incomplete or instructions unclear	Repository or deployment missing	CPA-3	2
Weighted CLO Score					
Remarks and Signature					

Lab Session 14

Introduction to Web Project Deployment and Maintenance Using GitHub Pages

1. Introduction

In modern software engineering, developing an application is only one stage of the **Software Development Life Cycle (SDLC)**. After the software has been designed, implemented, and tested, it must be **deployed** so that users can access and use it. Deployment refers to the process of making an application available in a real-world environment.

Deployment is a critical stage because it transforms the software from a local development project into a **publicly accessible system**. Without deployment, software cannot reach end users or provide practical value.

Once the application is deployed, developers must perform **software maintenance**. Maintenance involves updating the system to fix errors, improve functionality, enhance performance, and adapt to new requirements.

In web development, deployment typically means **hosting a website on a server** so that it becomes accessible through the internet using a public URL.

One of the easiest platforms for deploying static websites is **GitHub using its hosting service **GitHub Pages. GitHub Pages allows developers to publish websites directly from a repository without requiring a separate hosting provider.

In this lab, students will learn how to deploy an **existing web project** using GitHub Pages and then perform updates to simulate the **maintenance phase of SDLC**.

2. Learning Objectives

After completing this lab, students will be able to:

• Understand the concept of **software deployment**
• Understand the role of **software maintenance** in SDLC
• Understand the importance of **version control in deployment**
• Upload an existing project to GitHub
• Deploy a web application using GitHub Pages
• Access a deployed website through a public URL
• Perform updates and maintenance on a deployed application

3. Software and Tools Required

The following tools will be used in this lab:

Tool	Purpose
Git	Version control system used to track code changes
GitHub	Online platform for hosting repositories
GitHub Pages	Service used to deploy static websites
Visual Studio Code (optional)	Code editing
Web Browser	Accessing GitHub and deployed website

4. Theoretical Background

4.1 Software Development Life Cycle (SDLC)

The **Software Development Life Cycle** is a structured process used to develop high-quality software. It ensures that software is built in a systematic and organized manner.

Typical phases of SDLC include:

1. Requirement Analysis
2. System Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

In this lab, we focus on the **Deployment and Maintenance phases**.

4.2 Software Deployment

Deployment is the process of **releasing software to the production environment**, where it becomes accessible to users.

Deployment activities include:

• Uploading application files to a server
• Configuring hosting services
• Managing application versions
• Making the system available to users

In web development, deployment means **publishing a website online** so that it can be accessed using a web browser.

4.3 Software Maintenance

After software is deployed, it must be continuously updated and improved. This process is called **software maintenance**.

Software maintenance includes four major categories:

Corrective Maintenance

Fixing errors or bugs discovered after deployment.

Adaptive Maintenance

Updating the system to work with new environments or technologies.

Perfective Maintenance

Enhancing system functionality or performance.

Preventive Maintenance

Improving system design to prevent future problems.

In websites, maintenance often involves:

• Updating website content
• Fixing broken links
• Updating images or design
• Adding new pages

4.4 Version Control

Version control is a system that tracks changes in software code over time. It allows developers to manage different versions of the same project.

Git is one of the most widely used version control systems.

Advantages of version control:

• Tracks changes in project files
• Allows collaboration among developers
• Enables rollback to previous versions
• Supports project maintenance and updates

4.5 GitHub Platform

**GitHub is a cloud-based platform used for hosting Git repositories.

Developers use GitHub to:

• Store source code
• Manage project versions
• Collaborate with team members
• Deploy web projects

GitHub also provides **GitHub Pages**, which allows hosting websites directly from a repository.

4.6 GitHub Pages

**GitHub Pages is a static website hosting service provided by GitHub.

It allows developers to host websites using:

• HTML
• CSS
• JavaScript
• Images and assets

GitHub Pages automatically publishes website files stored in a repository and provides a **public URL**.

Example URL:

https://username.github.io/project-nam

e

5. Website Project Structure

Before deployment, the website should have a clear folder structure.

Example project structure:

website-project

```
├── index.html
├── about.html
├── contact.html
│
├── css
│   └── style.css
│
├── js
│   └── script.js
│
└── images
```

Important rule:

The main page of the website must be named:

index.html

This page will automatically load when users open the website.

6. Lab Procedure.

Step 1: Create a New Repository

1. Login to GitHub.
2. Click **New Repository**.
3. Enter repository name: website-deployment-lab
4. Choose **Public** repository.
5. Click **Create Repository**.

The repository will be created.

Step 2: Prepare Existing Website

Students will use an **already developed website project**.

Example files:

index.html
style.css
script.js
images

Ensure that the website works correctly when opened locally.

Step 4: Upload Website Files to GitHub

Method 1 (Using GitHub Interface)

1. Open your repository.
2. Click **Add File**.
3. Select **Upload Files**.
4. Drag and drop your website files.
5. Click **Commit Changes**.

The files will now appear in the repository.

Step 5: Enable GitHub Pages

1. Open the repository.
2. Click **Settings**.
3. Scroll down to **Pages**.
4. Under **Source**, select:

Deploy from branch

5. Choose:

Branch: main
Folder: /root

6. Click **Save**.

GitHub will begin deploying the website.

Step 6: Access the Deployed Website

GitHub will generate a public website URL:

https://username.github.io/website-deployment-la

b Open this link in your browser.

Your website will now be **live on the internet**.

7. Demonstrating Software Maintenance

After deployment, the website will be updated to simulate **maintenance activities**.

Step 1: Modify Website Content

Open **index.html** and modify the text.

Example:

<h1>Welcome to My Website</h1>

Change it to:

<h1>Welcome to My Updated Website</h1>

Save the file.

Step 2: Upload Updated Files

1. Open the GitHub repository.
2. Click **Add File → Upload Files**.
3. Upload the modified file.
4. Click **Commit Changes**.

Step 3: Verify Changes

Refresh the deployed website URL.

The updated content will appear on the website.

This demonstrates **software maintenance after deployment**.

8. Expected Output

After completing the lab:

• The website will be successfully deployed online
• Students will receive a public website URL
• The deployed website will load properly in the browser
• Updates made to files will appear on the live website

9. Advantages of Using GitHub Pages

GitHub Pages offers several benefits:

• Free website hosting
• Easy deployment process
• Integration with version control
• Global content delivery network
• Suitable for portfolios and project hosting

It is widely used by developers and students to publish web projects.

EXERCISE

Students are required to take an existing static website project containing HTML, CSS, and related assets and deploy it online using GitHub and its hosting service GitHub Pages. After successful deployment, perform a maintenance activity by modifying some part of the website (such as updating text, changing images, or adding a small section) and upload the updated files to the repository so that the deployed website reflects the changes.

Students must submit the following:

1. GitHub Repository Link
2. Live Website URL
3. Screenshot of the deployed website
4. Screenshot after performing maintenance update